Algoritmos e Complexidades

Holy Grail

Resolução dos testes/exames + Notas de estudo

91666 (a‘_‘a)

2025/2026

Resumo

Este pdf contem algumas resoluções dos testes e exames de algoritmos e notas de estudo (**RESOLVIDOS POR MIM !**).

- Exames de 22/23 até 24/25 ;
- Testes de 22/23 até 24/25 ;
- Notas de estudo de Algoritmos .

Disclaimer

Importante relembrar que as resoluções podem estar incorretas !
Não considerem tudo que está neste pdf como verdade absoluta, tenham sempre em atenção o que foi dado nas aulas Teóricas e Teóricas-Práticas e qualquer dúvida perguntem aos docentes responsáveis pela UC :D

Índice

1 Exame 24/25 **4**

1.1 I - Correção de Soma2a2 . . . 4
1.2 II - Complexidade de Soma2a2 . . . 5
1.3 II - Relação de Recorrência . . . 6
1.4 III - AVL . . . 7
1.5 III - AVL (Fator de Balanço) . . . 9
1.6 III - THash . . . 9
1.7 III - THash (Quantas posições) . . . 10
1.8 III - THash (Inserção) . . . 11
1.9 III - Grafos . . . 12
1.10 III - THash (Dijkstra) . . . 13
1.11 IV - MiniHeap . . . 13
1.12 IV - MiniHeap (Complexidade) . . . 14
1.13 V - Grafos . . . 15
1.14 V - Grafos (Complexidade) . . . 16

2 Exame 23/24 **17**

2.1 I - Especificação e Correção (1) . . . 17
2.2 I - Especificação e Correção (2) . . . 18
2.3 II - Complexidade algoritmos iterativos e recursivos (1) . . . 18
2.4 II - Complexidade algoritmos iterativos e recursivos (2) . . . 19
2.5 III - Heaps . . . 20
2.6 IV - Grafos (a) . . . 21
2.7 IV - Grafos (b) . . . 22
2.8 V - Grafos . . . 23

3 Exame 22/23 **26**

3.1 I - Correção (1) . . . 26
3.2 I - Correção (2) . . . 27
3.3 II - Complexidade . . . 27
3.4 III - Tabelas de Hash . . . 29
3.5 IV - Análise amortizada (1) . . . 30
3.6 IV - Análise amortizada (2) . . . 30

3.7 V - Grafos . . . 31

4 Teste 1 - 22/23 **34**
4.1 I - Especificação . . . 34
4.2 II - Correção . . . 34
4.3 III - Complexidade algoritmos iterativos . . . 35
4.4 IV - Complexidade algoritmos recursivos . . . 36

5 Teste 2 - 22/23 **40**
5.1 I - Árvores AVL . . . 40
5.2 II - Análise de caso médio . . . 41
5.3 III - Min-heaps . . . 43
5.4 IV -Grafos . . . 44

6 Teste 1 - 23/24 **46**
6.1 I - Especificação . . . 46
6.2 II - Correção . . . 47
6.3 III - Complexidade algoritmos iterativos . . . 47
6.4 IV - Complexidade de algoritmos recursivos . . . 49

7 Teste 2 - 23/24 **51**
7.1 I - Tabelas de Hash . . . 51
7.2 II - Árvores Binárias / AVL . . . 53
7.3 III - Grafos . . . 55
7.4 IV - Grafos . . . 57

8 Teste 1 - 24/25 **59**
8.1 I - Especificação . . . 59
8.2 II - Correção . . . 60
8.3 III - Complexidade de algoritmos recursivos . . . 60
8.4 IV - Complexidade algoritmos iterativos . . . 62

9 Teste 2 - 24/25 **65**
9.1 I - Estruturas de Dados . . . 65
9.2 II - Tabelas de Hash . . . 72
9.3 III - Min-heap . . . 73
9.4 IV - Grafos . . . 74

A Notas de estudo **76**
A.1 Pré-Condição . . . 76
A.2 Pós-Condição . . . 76
A.3 Invariante . . . 77
A.4 Variante . . . 77
A.5 Complexidade . . . 77

A.5.1 Recursividade . . . 78
A.5.2 Pior Caso . . . 78
A.5.3 Melhor Caso . . . 78
A.5.4 Caso Medio . . . 78
A.5.5 Custo Real vs Custo Armotizado . . . 80
A.5.6 Custo Amortizado . . . 80
A.6 Hash Tables . . . 81
A.6.1 Garbage Collection . . . 82
A.6.2 Inserção na Hash Table . . . 82
A.7 Árvores . . . 83
A.7.1 Árvore Binária de Pesquisa (BST) . . . 83
A.7.2 Heap(min/max) . . . 83
A.7.3 Min-Heap . . . 83
A.7.4 Árvore equilibrada (AVL) . . . 84
A.7.5 Array para Árvore . . . 85
A.7.6 ABin . . . 85
A.7.7 Rotações da Árvore . . . 85
A.8 Algoritmos de Pesquisa . . . 86
A.8.1 Dijkstra's . . . 86
A.8.2 Prim's . . . 87
A.8.3 BFS . . . 88
A.8.4 DFS . . . 88
A.8.5 DFS vs BFS . . . 88
A.8.6 Notas de Grafos . . . 88
A.9 Notas Extra . . . 89
A.9.1 Fibonacci . . . 89

Capítulo 1

Exame 24/25

1.1 I - Correção de Soma2a2

Considere a definição da seguinte funçao que **comprime um array** somando os seus elementeos dois a dois.

```
1  int soma2a2(int v[ ], int N) {
2      //Pre: N > 0
3      int r = 1, w = 0;
4      while(r < N){
5          //Inv: ?? Var: ??
6          v[w] = v[r] + v[r-1];
7          w += 1; r += 2;
8      }
9      if(N % 2 == 1) v[w++] = v[r-1];
10     //Pos: w == (N+1) \ 2
11     return w;
12 }
```

Apresente um invariante e variante que lhe permitam provar a correção total da função. Note que a pós-condição apresnetada **não é a pós-condição do ciclo**.
Sugestão: Comece por escrever a pós-condiçao do ciclo

Resolução:

A cada iteração do ciclo:

- r aumenta 2 ;
- w aumenta 1 ;
- já foram calculadas as somas dos pares anteriores .

Importante saber que:

- w = nº de elementos depois da soma ou seja $w = \frac{(N+1)}{2}$;
- r = identificador para fazer os pares .

Logo o **Invariante** seria:

$$r = 2w + 1 \wedge 0 \leq w \leq (N+1)/2 \wedge \forall_{0 \leq i < w}, v[i] = v[2i] + v[2i+1]$$

Esta parte do invariante $\underline{\forall_{0 \leq i < w}, v[i] = v[2i] + v[2i+1]}$ *representa todas as posições já escritas (**somas calculadas**)*

O invariante permite provar que, quando o ciclo termina, todas as somas dos pares foram feitas. Se N for impar, o último elemento é copiado.

O **Variante** seria:

$$N - r$$

é sempre positivo durante o cíclo e diminui 2 em cada iteração, provando a **terminação**.

1.2 II - Complexidade de Soma2a2

Analise a complexidade da função soma2a2 (da questão I) em termos do **nº de acessos de leitura e escrita no array.**

```
1 int soma2a2(int v[ ], int N) {
2     int r = 1, w = 0;
3     while(r < N){
4         v[w] = v[r] + v[r-1];
5         w += 1; r += 2;
6     }
7     if(N % 2 == 1) v[w++] = v[r-1];
8     return w;
9 }
```

Resolução:

Em cada iteração de ciclo temos:

- 2 leituras $v[r]$ e $v[r-1]$;
- 1 escrita $v[w]$.

Para alem disso é importante notar que o ciclo executa:

- $\frac{N}{2}$ iterações se **N par** ;
- $\frac{N-1}{2}$ iterações se **N impar** .

Em termos de $\underline{\text{nº total de leituras}}$:

$$\text{Se N par: } 2 * \frac{N}{2} = N$$

$$\text{Se N impar: } 2 * \frac{N-1}{2} + 1 = N - 1 + 1 = N$$

- $2 * \frac{N-1}{2}$ corresponde às leituras no ciclo
  - 1 leitura na condição do if

Logo a complexidade em termos de leitura θ(N).

Em termos de nºtotal de escrita:

$$\text{Se N par: } 1 * \frac{N}{2} = \frac{N}{2}$$

$$\text{Se N impar: } 1 * \frac{N-1}{2} + 1 = \frac{N-1}{2} + 1 = \frac{N-1+2}{2} = \frac{N-1}{2}$$

- $1 * \frac{N-1}{2}$ corresponde às escritas no ciclo
  - 1 escrita na condição do if

Logo a complexidade em termos de escrita θ(N).

Nota: Em caso de pedir só Assintótica, basta apenas escrever leitura θ(N) e escrita θ(N).

1.3 II - Relação de Recorrência

Considere agora a definição da função **soma** que usa a função da questão anterior:

```
1 int soma( int v[ ], int N){
2     int t;
3     if(N == 1) return v[0];//basicamente, soma ate so ter 1 elemento e devolve-o AKA
          o total.
4     t = soma2a2( v, N);
5     return some(v,t);
6 }
```

Apresente e resolva uma **relação de recorrência** que traduza a complexidade da função soma(mais uma vez tendo em conta **apenas o nº de acessos ao array**)

Resolução:

Sabemos da questão anterior que **soma2a2(v,N)** faz θ(N) acessos ao array (leitura + escrita).

Além disso, após cada chama: $T = \frac{N+1}{2} \approx \frac{N}{2}$

Se T(N) representa o nº de acessos ao array:

Caso Base:

$$T(1) = 1 \text{ (apenas acede uma vez para dar return do v[0])}$$

Para $N > 1$:

$$T(N) = T(\tfrac{N+1}{2}) + N \text{ (o que acontece cada vez que executa)}$$

- $T(\frac{N+1}{2})$ corresponde à recorrencia

- N corresponde à função soma2a2

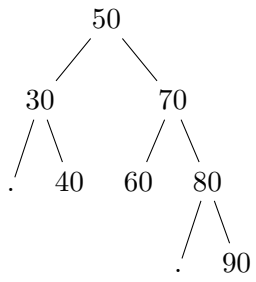
Como $T = \frac{N+1}{2} \approx \frac{N}{2}$

$$\begin{aligned} T(N) &= T(\frac{N}{2}) + N \\ &= N + T(\frac{N}{2}) \\ &= N + \frac{N}{2} + T(\frac{N}{4}) \\ &= N + \frac{N}{2} + \frac{N}{4} + ... = N(1 + \frac{N}{2} + \frac{N}{4} + ...) \end{aligned} \tag{1.1}$$

Como $1 + \frac{N}{2} + \frac{N}{4} + ... = 2$ então $\underline{T(N) \leq 2N}$

$$\text{Logo } T(N) = \theta(N)$$

1.4 III - AVL



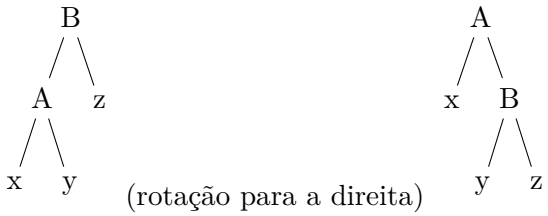
Qaul o **valor da raíz da árvore** depois de serem inseridas as chaves 65, 85 e 20 (por esta ordem)?

Resolução:

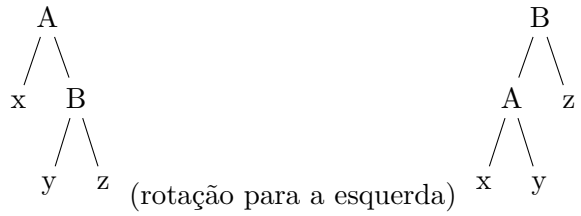
Nota Importante:*(Os nodos com ponto são apenas para representação dos lados da árvore* ***vazios****, não é preciso desenhar no papel.)*

Nota: Cada nodo/raiz sera representado por $Raiz^{Altura/Fator\ Balanco}$, onde sempre que o Fator de balanço for diferente de $-1, 0, 1$, significa que temos de fazer rotação nesse nodo/raiz.

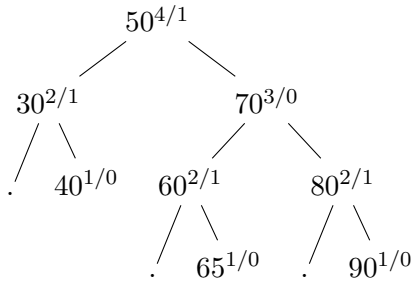
Left $->$ Right



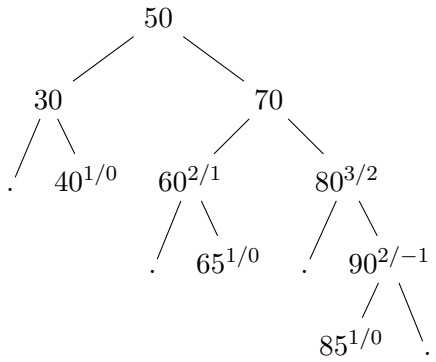
Right $->$ Left



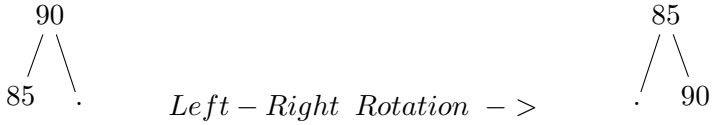
Inserir 65:

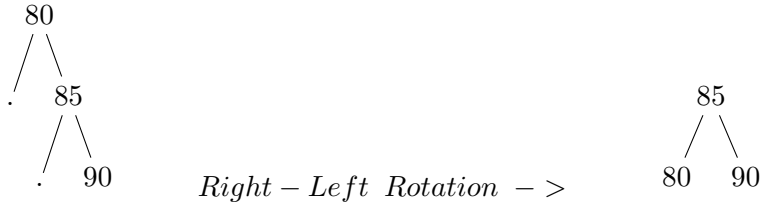


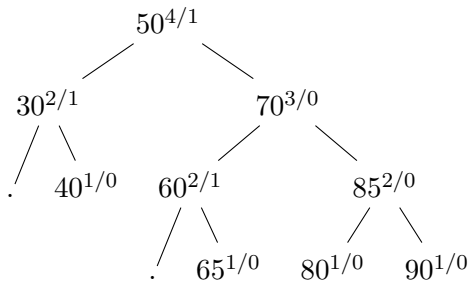
Inserir 85:



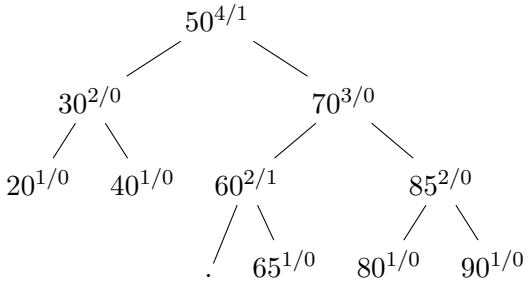
A árvore ficou desbalancia no nodo 80. Vamos ter de fazer **dupla rotação**, primeira right rotation e depois uma left rotation.







Inserir 20:



1.5 III - AVL (Fator de Balanço)

Qual o factor de balanço(Esq,Bal ou Dir) da raíz da árvore depois dessas inserções?

Resolução:

Fator de bananço $-> altura\ direita - altura\ esquerda = [-1, 0, 1]$

No caso da árvore deste exercício $3 - 2 = 1$ logo a raíz está inclinada para a direita, Dir.

1.6 III - THash

Considere a seguinte tabela de hash de inteiros, implementada com open-addressing e linear probing num array com 10 elementos hash é o resto da divisão inteira ($hash(x) = x\%10$) e o estado de cada célula é FREE, DELETE ou USED.

0	1	2	3	4	5	6	7	8	9
D	U	D	D	U	U	F	D	U	U
10	70	32	12	54	43	65	17	27	38

Após uma garbage collection em que posição ficará a chave 43?

Resolução:

Garbage Collection $->$ faz os DELETE desaparecerem e re-insere todos os USED num array de dimensões iguais ao atual(insere por ordem que aparecem).

$Used = 70, 54, 43, 27, 38$

Tendo em consideração ($hash(x) = x\%10$) as novas posições serão:

- $70 = 10 * 7 + \mathbf{0}$, posição 0
- $54 = 10 * 5 + \mathbf{4}$, posição 4
- $43 = 10 * 4 + \mathbf{3}$, posição 3

(Extras)

- $27 = 10 * 2 + \mathbf{7}$, posição 7
- $38 = 10 * 3 + \mathbf{8}$, posição 8

0	1	2	3	4	5	6	7	8	9
U 70	F	F	U 43	U 54	F	F	U 27	U 38	F

Logo 43 fica na posição 3.

1.7 III - THash (Quantas posições)

Após essa operação de garbage collection foi adicionada uma chave (aleatória) entre 100 e 199 (inclusive). Em média, quantas posições do array consulta essa inserção?

Resolução:

- Entre 100 e 199 há exatamente 10 números para cada resto, onde cada posição (0 a 9) é igualmente provável ;
- Temos 5 posições ocupadas e 5 posições livres .

0	1	2	3	4	5	6	7	8	9
U 70	F	F	U 43	U 54	F	F	U 27	U 38	F

Vamos agora analisar quantas posições são consultadas para inserir a nova chave:

Se começar na posição...

- **0** $->$ ocupado, 1 $->$ livre = Analisa **2 posições**;

- **1** $->$ livre = Analisa **1 posição**;
- **2** $->$ livre = Analisa **1 posição**;

- **3** $->$ ocupado, 4 $->$ ocupado, 5 $->$ livre = Analisa **3 posições**;
- **4** $->$ ocupado, 5 $->$ livre = Analisa **2 posições**;
- **5** $->$ livre = Analisa **1 posição**;
- **6** $->$ livre = Analisa **1 posição**;

- **7** $->$ ocupado, 8 $->$ ocupado, 9 $->$ livre = Analisa **3 posições**;
- **8** $->$ ocupado, 9 $->$ livre = Analisa **2 posições**;
- **9** $->$ livre = Analisa **1 posição**;

Assim a média será calculada da seguinte forma:

$$M = \frac{2+1+1+3+2+1+1+3+2+1}{10\ (posicoes)} = \frac{17}{10} = 1.7$$

Em média, são consultadas **1.7** posições do array.

1.8 III - THash (Inserção)

Após a operação de garbage collection referida na alínea 1. Foram adicionadas as chaves 47 e 48 (por esta ordem). Em que posição fica armazenada a chave 47 ?

Resolução:

Tendo em consideração ($hash(x) = x\%10$) as novas posições serão:

- $47 = 10 * 4 + \mathbf{7}$, posição 7
- $18 = 10 * 1 + \mathbf{8}$, posição 8

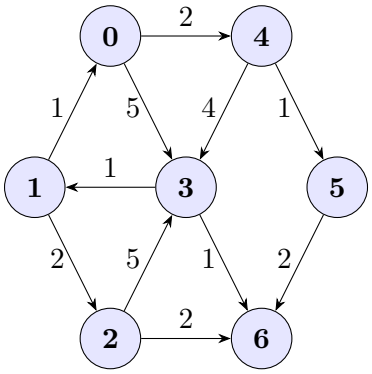
Nota: Se a nova chave calhar numa posição que já esta ocupada, avança para a proximo (ou volta para o início do array, caso seja a última célula) até encontrar uma posição **FREE**.

0	1	2	3	4	5	6	7	8	9
U	U	F	U	U	F	F	U	U	U
70	18		43	54			27	38	47

Logo a chave 47 encontra-se na **posição 9**.

1.9 III - Grafos

Seja g o seguinte grafo pesado e orientado representado usando listas de adjacência ordenadas por **ordem crescente** do destino.



Numa travessia Depth-First iniciado no vértice 4, qual o último vértice a ser visitado ?

Resolução:

Nota: Depth-First, não tem interesse se o grafo é pesado, só temos de ter em consideração os vizinhos que temos no nodo onde estamos e respeitar o **sentido** das arestas.

Lista de adjacência (começando em **4**)

{4} , DFS={}

{3, 5} − > {~~3~~, 5} (seguimos para o nodo 3), DFS={4}

{1, 6} − > {~~1~~, 6} (seguimos para o nodo 1), DFS={4,3}

{0, 2} − > {~~0~~, 2} (seguimos para o nodo 0), DFS={4,3,1}

{3, 4} − > {~~3~~, ~~4~~} (ambos já foram visitados anteriormente, e por isso voltamos para o par anteriror), DFS={4,3,1,0}

{~~0~~, 2} − > {~~0~~, ~~2~~} (seguimos para o nodo 2), DFS={4,3,1,0}

{3, 6} − > {~~3~~, ~~6~~} (o 3 já tinha sido vesitado anteriormente, logo vamos visitar o 6), DFS={4,3,1,0,2}

{} − > {~~6~~} (o vertice 6 não tem arestas a sair para outro nodo, por isso é acrescentado à lista e voltamos para o par anterior onde pode haver ainda algum vertice por visitar, no caso o 5), DFS={4,3,1,0,2,6}

{3, 5} − > {~~3~~, ~~5~~} (Terminamos de explorar todos os vertices do grafo), DFS={4,3,1,0,2,6,5}

Logo o último a ser visitado é o vértice **5**.

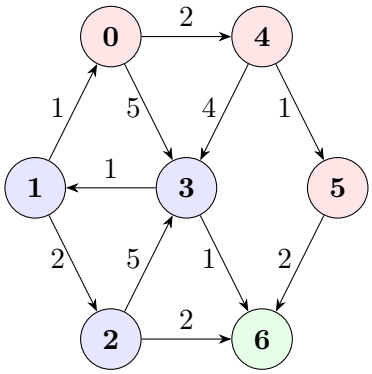
1.10 III - THash (Dijkstra)

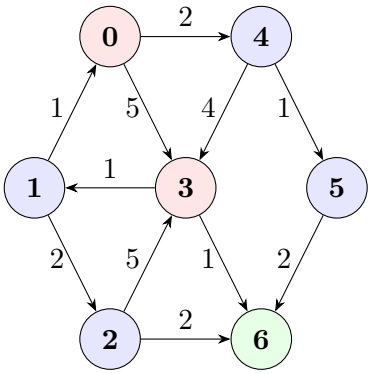
Após a execução de **dijkstraSP(g, 0 ,pais, peso)** qual o valor de pais[6]?

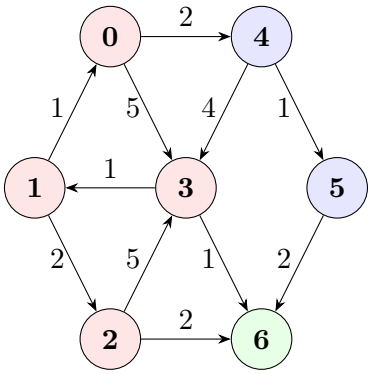
Resolução:

- Tendo em consideração a função dijkstraSP, sabemos que o vértice inicial é 0 ;
- $pais[k]$ é o vertice anterior do k ;
- A partir de 0 vamos calcular o melhor caminho no grafo até 6, e $pais[6]$ é o vertice anterior a 6.
- O algoritmo de Dijkstra tem em consideração os pesos do grafo, vai somando os valores até percorrer o grafo inteiro, sendo o melhor caminho, o caminho com menor valor. Ou seja, neste caso vamos calcular o caminho com o menor valor até 6, partindo de 0.

Do vertice 0 a 6 temos 3 opções de caminhos:







$Caminho = 2 + 1 + 2 = 5$ $Caminho = 5 + 1 = 6$ $Caminho = 5 + 1 + 2 + 2 = 10$

De todos os caminhos, o melhor caminho corresponde ao **primeiro grafo**, uma vez que o peso total final é 5.

Logo o $pais[6] = 5$.

1.11 IV - MiniHeap

Defina uma função $ABin\ \ fromArray\ (int\ h[\,], int\ N)$ que constrói a árvore binaria correspondente à mini-heap armazenada num array h com N elementos.

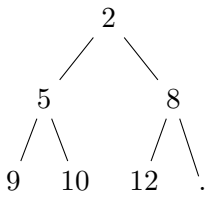
Resolução:

Mini-heap [Array] − > Arvore Binaria

- Numa mini-heap raiz $\leq$ todos os outros nodos ;

- Como a mini-heap está armazenada num array, sabemos que a poisção 0 é a raiz e para aceder ao resto:
  - lado esquerdo $i \ -> \ 2*i+1$;
  - lado direito $i \ -> \ 2*i+2$.

Exemplo visual:



Indice	0	1	2	3	4	5
Valor	2	5	8	9	10	12

```
typedef struct nodo{
    int valor;
    struct nodo *esq;
    struct nodo *dir;
}*Abin

Abin aux(int h[], int N, int i){
    //int i -> cada indice i representa a raiz de uma subarvore
    if (i >= N)
        return Null;
    Abin novo = malloc(sizeof(struct nodo));
    novo->valor = h[i];
    novo->esq = aux(h,N,2*i+1);
    novo->dir = aux(h,N,2*i+2);
    return novo;
}

Abin fromArray (int h[], int N){
    return aux(h,N,0);
}
```

1.12 IV - MiniHeap (Complexidade)

Analise a complexidade da definição apresentada.

Resolução:

Cada chamada **aux** faz apenas trabalho constante :

comparação = 1

malloc = 1

atribuir valor = 1

duas chamadas (lado esq e lado dir) = 1

Logo $O(1)$. Como existem N nós, entao $T(N) = N * O(1) = \mathbf{O(N)}$

1.13 V - Grafos

Defina uma função *int middle_point* (*GrafoL g*, *int a*, *int b*) que, dado um grafo (não pesado e orientado) e dois vertices, calcula o vértice v que minimiza a soma das distâncias de ***a*** a ***v*** e de ***b*** a ***v***. A função retorna **-1** se não existir tal vertice. Use se precisar as funções estudadas nas aulas TP. Para as funções que usar, comece por explicar o seu tipo.

Resolução:

- $dist\ (a, 4) + dist\ (b, v)$, queremos o "ponto de encontro" para os vertices **a** e **b** ;
- Imaginemos que $dist\ (a, 4) = 2$ e $dist\ (b, 4) = 1$. Então o custo do vértice 4 é $2 + 1 = 3$;
- Agora temos de cálcuar para <u>todos os vertices</u> ;
- **Grafo não pesado e orientado $->$ BFS**

Nota : *typedef GrafoL*$[NV]$, onde NV é o <u>nº de vertices</u>.

```c
int middle_point (GrafoL g, int a, int b){
    int distA[MAX];
    int distB[MAX];

    bfs(g,a,distA);
    bfs(g,b,distB);

    int melhor = -1;
    int min = INT_MAX;
    //daria para fazer de outra forma, mas neste caso INT_MAX = +∞
    //entao para comecar, qualquer valor vai ser < que +∞
    for(int i = 0; i < NV; i++){
        if(distA[i] != -1 && distB[i] != -1){
            if(distA[i] + distB[i] < min){
                min = distA[i] + distB[i];
                melhor = i;
            }
        }
    }
    return melhor;
}
```

1.14 V - Grafos (Complexidade)

Qual a complexidade da função apresentada ?

Resolução:

Sabemos que:

- **BFS** $-> O(V+E)$;
- Como são $2 * BFS \ -> \ 2 * (V+E)$;
- Percoerrer os vertices $-> O(V)$.

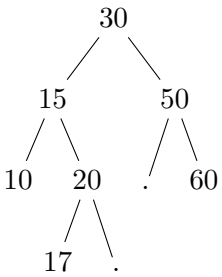
Logo a complexidade da função é **O(V+E)** .

Capítulo 2

Exame 23/24

2.1 I - Especificação e Correção (1)

Recorde que as estruturas de dados conhecidas como heaps são árvores binárias implementadas fisicamente sobre arrays. É possível implementar quaisquer outras árvores binárias sobre arrays, bastando para isso utilizar um valor especial para representar as lacunas, i.e. posições da árvore em que não se encontra qualquer nó. Por exemplo a seguinte árvore de números inteiros



pode ser representada por arrays preenchidos desta forma:

| 30 | 15 | 50 | 10 | 20 | H | 60 | H | H | 17 | ... H ... |

Considere agora a seguinte função de procura numa árvore binária (BST) representa desta forma:

```
int bsearch (int t [], int x, int N){
    //Pre: ...
    int i = 0;
    while (i < N && a[i] != H && a[i] != x)
        if (a[i] > x) i = 2*i+1; else i = 2*i+2;
    if (I == N || a[i] == H) i = -1;
    //Pos: ...
    return i;
}
```

Escreva uma **pré-condição** e uma **pós-condição** que descrevam o seu comportamento.

Resolução:

Pre : $N \geq 0 \quad \& \ \& \quad a[0 \ldots N-1]$ representa uma BST válida

Pos : $(i \neq -1 \implies 0 \leq i < N \quad \& \ \& \quad a[i] = x) \ || \ (i == -1 \implies \forall_{x, 0 \leq k < N}, \ a[k] \neq x)$

2.2 I - Especificação e Correção (2)

Escreva um invariante e um variante para o ciclo, que lhe permitam provar a correção total da função. No caso do invariante se não souber especificá-lo formalmente explique o mesmo "por palavras".

Resolução:

Em cada iteração:

- i é o índice do nó atual ;
- Se x for menor, desce para o filho esquerdo ;
- Se x for maior, desce para o filho direito .

Logo a cada iteração continuamos apenas na única subarvore onde x ainda pode estar.

Invariante :

- Antes de cada iteração, se x existir na BST, então encontra-se na subarvore enraizada em i.

$$\exists_{0 \leq k < N}, \ a[2 * k + 1] == x \ || \ a[2 * k + 2] == x$$

Vairante : nº de nós da subarvore enraizada em i ou N-i(variante frágil)

2.3 II - Complexidade algoritmos iterativos e recursivos (1)

Observe agora a seguinte versão recursiva da função da Questão 1:

```
1 int bsearch (int t [], int i, int x, int N){
2     if (i >= N || a[i] == H) return -1;
3     if (a[i] > x) return bsearch (t, 2*i+1, x, N);
4     if (a[i] < x) return bsearch (t, 2*i+2, x, N);
5     return i;
6 }
```

Identifique o melhor e o pior caso do seu tempo de execução em função de N (o tamanho do array).
Escreva, e resolva, recorrências para esses dois casos, contando as operações de comparação (a[i]>x) e (a[i]<x).

Resolução:

O **melhor caso** acontece quando encontramos x na primeira chama, ou seja $a[0] == x$.

- Tendo em consideração as comparações feitas ao executar *bsearch*, uma vez que são o foco :
  - $a[i] > x$ tem um tempo constante $= 1$;
  - $a[i] < x$ tem um tempo constante $= 1$.
- Como são feitas 2 comparações por execução, então $\underline{T(1) = 1 + 1 = 2 = \theta(1)}$.

O **pior caso** acontece se x não existir ou está na folha mais profunda da árvore.

- Em cada chamada $if\ (a[i] > x)$ e $if\ (a[i] < x)$ são ambas avaliadas ;
- Como acontecem 2 comparações por nível da árvore, a cada chamada descemos para apenas um filho, ou seja :

$$T(N) = T(\frac{N}{2}) + 2$$

(onde $T(\frac{N}{2})$ é o próximo filho que vamos verificar e 2 é o número de comparações que fizemos na chamada)

- Desta forma a altura da árvore reduz aproximadamente para metade.

Sendo assim teremos:

Caso Base:

$$T(1) = 2$$

Para $N \geq 1$:

$$\begin{aligned} T(N) &= 2 + T(\frac{N}{2}) \\ &= 2 + 2 + T(\frac{N}{4}) \\ &= 2 + 2 + 2 + ... \end{aligned} \tag{2.1}$$

- A profundidade da árvore é $\underline{\log_2 N}$;
- Se existem $\log_2 N$ níveis e a cada nível acontece 2 comparações, então $2 + 2 + ... + 2\ \log_2 N$ (vezes)$= 2 * \log_2 N$.

$$\text{Assim } \underline{T(N) = 2 * \log_2 N + 2 = \theta(\log_2 N)}$$

2.4 II - Complexidade algoritmos iterativos e recursivos (2)

Considerando o caso particular das árvores de procura representadas por arrays sem ocorrências de H (i.e., árvores completas, em que o comprimento do array coincide com o número de elementos da árvore) e assumindo que x se encontra na árvore, analise o tempo de execução da função de procura no **caso médio**.

N.B. Poderá basear a sua análise em qualquer uma das duas versões da função.

Resolução:

Numa **árvore completa**, a altura é $\theta(\log N)$. Como o elemento procurado existe sempre e a profundidade média de um nó é proporcional à altura da árvore, o tempo de execução no caso médio é $\underline{\theta(\log N)}$.

2.5 III - Heaps

Considere uma *min-heap* inicialmente vazia, implementada sobre um *array dinãmico* com comprimento inicial igual a 1 (e que duplica de tamanho quando está cheio), e a seguinte sequência de operações sobre ela :

Insert 8 ;
Insert 18 ;
Insert 7 ;
Insert 10 ;
ExtractMin ;
Insert 29 ;
Insert 3 ;
ExtractMin ;
Insert 20 ;

Apresente os arrays correspondentes aos estados da heap em todos os passos desta sequência.

Resolução:

Insert 8 : $[8]$
Insert 18 : $[8, 18]$ (duplicou)
Insert 7 : $[8, 18, 7, __] \; -> \; [7, 18, 8, __]$ (duplicou)
Insert 10 : $[7, 18, 8, 10] \; -> \; [7, 10, 8, 18]$
ExtractMin : $[8, 10, 18, __]$
Insert 29 : $[8, 10, 18, 29]$
Insert 3 : $[8, 10, 18, 29, 3, __, __, __] \; -> \; [3, 8, 18, 29, 10, __, __, __]$ (duplicou)
ExtractMin : $[8, 10, 18, 29, __, __, __, __]$
Insert 20 : $[8, 10, 18, 29, 20, __, __, __]$

2.6 IV - Grafos (a)

Considere o grafo não orientado, com pesos, representado pela seguinte matriz de adjacências (peso 0 significa que não há aresta entre os nós respectivos) :

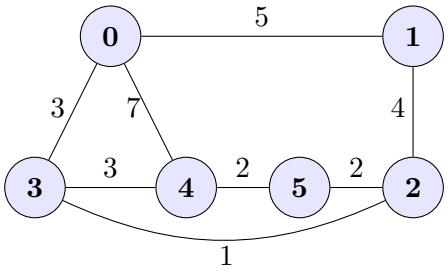
0	5	0	3	7	0
5	0	4	0	0	0
0	4	0	1	0	2
3	0	1	0	3	0
7	0	0	3	0	2
0	0	2	0	2	0

Desenhe o grafo correspondente a esta matriz de adjacências.

Resolução:

Vértices	0	1	2	3	4	5
0	0	5	0	3	7	0
1	5	0	4	0	0	0
2	0	4	0	1	0	2
3	3	0	1	0	3	0
4	7	0	0	3	0	2
5	0	0	2	0	2	0

(Onde tem 0 significa que **nao tem aresta** a ligar os vertices)



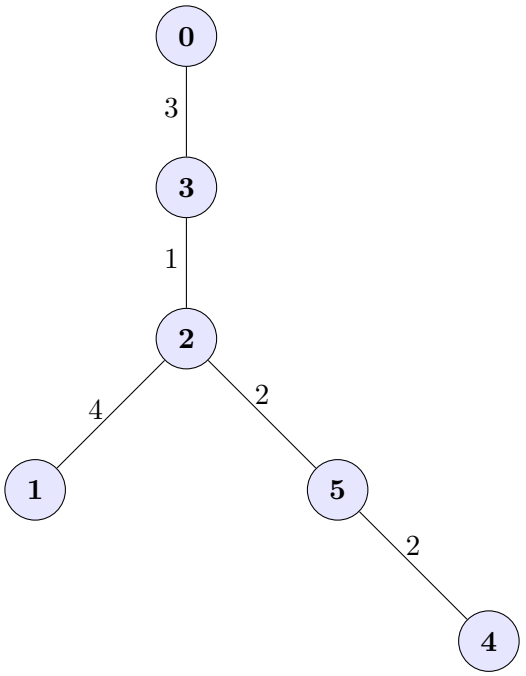
2.7 IV - Grafos (b)

Apresentando todos os passos, aplique o algoritmo de Prim para construir uma árvore geradora mínima do grafo, começando no **nó 0**.

Resolução:

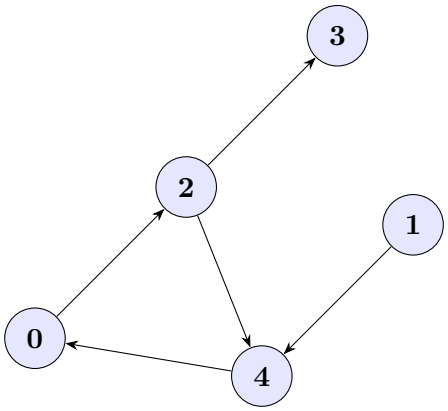
Nota : Algoritmo de Prim, opta por escolher o caminho menos pesado (se tiver dois com o mesmo peso a escolha é arbritária) .

Passo	Vértices na árvore	Aresta escolhida
1	$\{0\}$	$0-3 \ \ (peso = 3)$
2	$\{0, 3\}$	$3-2 \ \ (peso = 1)$
3	$\{0, 3, 2\}$	$2-5 \ \ (peso = 2)$
4	$\{0, 3, 2, 5\}$	$5-4 \ \ (peso = 2)$
5	$\{0, 3, 2, 5, 4, 1\}$	$2-1 \ \ (peso = 4)$
		$Total = 12 \ \ peso$



2.8 V - Grafos

Escreva uma função que detecta se um vértice x de um grafo orientado faz parte de um ciclo. Por exemplo, para o grafo seguinte a função deverá devolver 0 (falso) para os vértices 1 e 3, e 1 para todos os restantes vértices.




```c
struct edge { int dest; struct edge *next; };
typedef struct edge *GrafoL[MAX];

int in_cycle (GrafoL g, int x){
    ...
}
```

Analise o tempo de execução da função que escreveu, identificando o melhor e pior caso.

Serão valorizadas as soluções mais eficientes.

Resolução:

Ideia princípal:

- 1º – Começar em x ;
- 2º – Fazer uma DFS (ou BFS) ;
- 3º – Se em algum momento voltar a encontrar x, devolvemos 1 ;
- 4º – Se terminar a pesquisa sem voltar ao x, devolvemos 0 .

```c
struct edge { int dest; struct edge *next; };
typedef struct edge *GrafoL[MAX];

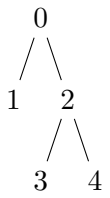
int dfs(GrafoL g, int origem, int v, int vis[ ]){

    //marca o vertice que vamos verificar, como vizitado, uma vez que comeca nele
    vis[v] = 1;

    //percorre todas as arestas que saem do vertice v
    for(struct edge *e = g[v]; e != NULL; e = e -> next){
```

```
11
12          if(e -> dest == origem)
13              //se um dos vizinhos for o vertice onde comecamos
14              return 1;
15
16          if(!vis[e->dest]){
17              //se ainda nao visitei este vertice, continua a DFS
18
19              if(dfs(g, origem, e->dest, vis))
20                  //esta chamada tanto pode devolver 0 ou 1, se devolver
21                  //1 significa que mais abaixo na arvore foi encontrado
22                  //um ciclo "se um dos meus filhos encontrou 1 tambem devolvo 1"
23                  return 1;
24          }
25
26      }
27      return 0;
28      //percorreu todos os caminhos mas nunca conseguiu voltar ao vertice inicial
29  }
30  int in_cycle (GrafoL g, int x){
31      int vis[MAX] = {0}; //vetor para guardar vizitados
32      return dfs(g, x, x, vis);//o primeiro x guarda a origem(ancora)
33                               //e o segundo vai mudando
34  }
```

Para entender melhor o que está a acontecer :



- O **for** vai percorrer os vizinhos de 0 :
  - $1^{o} -> 1$
  - $2^{o} -> 2$
- O for apenas **aponta** para eles, não os visita .
- **for** $->$ "Quais são os meus vizinhos ?"
- **recursão** $->$ "Agora que escolhi um vizinho, o que existe a partir dele ?"
- Dito de outra forma :
  - **for** olha à volta e vê as "portas disponíveis"(**escolhe**) ;
  - **recursão** "abre uma dessas portas e percorre todo o corredor até ao fim". Quando regressa, o **for** "escolhe a porta seguinte"(**explora**) .

Relativamente ao tempo de execução :

- Como temos o vetor vis, cada vertice é marcada uma vez. Por exemplo, se mais tarde existir outra aresta para o vertice 2, não voltamos a explora-lo. Logo cada vertice é visitado uma vez $O(V)$;
- $for(\ e = g[V];\ e\ != Null;\ e = e->next)$, este cíclo percorre todas as arestas que saem de v. Como nenhum vértice é visistado duas vezes, cada lista de adjacência é percorrida apenas uma vez, logo $O(E)$.

Como cada vértice é visitado no máximo uma vez devido ao vetor de visitados, e cada aresta é analisada no máximo uma vez ao percorrer as listas de adjacência. Assim, o tempo de execução é proporcional ao nº de vertices + nº de arestas do grafo, ou seja:

$$\theta(V + E)$$

Capítulo 3

Exame 22/23

3.1 I - Correção (1)

A função *prefSum* calcula o comprimento do **maior prefixo** de um array cuja soma não excede um determinado valor.

```
1  int prefSum (int v[ ], int N, int lim){
2      //pre: N >= 0 && lim >= 0
3      int sum = 0, i = 0;
4      while (i < N && sum + v[i] <= lim){
5          //inv: ??
6          sum = sum + v[i]; i = i + 1;
7      }
8      //pos: ??
9      return i;
10 }
```

1. Complete a especificação abaixo fornecendo uma pós-condição apropriada.

Resolução:

A soma dos primeiros i elementos é menor ou igual ao limite :

$$\sum_{k=0}^{i-1} v[k] \leq lim$$

(onde $i-1$ representa quantos elementos já foram somados)

Não é possível acrescentar o elemento seguinte (exceto se chegarmos ao fim), logo :

$$i = N \;\; ou \;\; \sum_{k=0}^{i} v[k] > lim$$

(onde i representa o próximo elemento que vai ser somado)

Assim, a pós-condição :

$$\sum_{k=0}^{i-1} v[k] \leq lim \;\wedge\; (i = N \;\; \vee \;\; \sum_{k=0}^{i} v[k] > lim \;)$$

- i é um contador de elementos :
  - $i = 0 \ -> \ 0$ elementos ;
  - $i = 1 \ -> \ 0 + \text{v}[0]$;
  - $i = 2 \ -> \ \text{v}[0] + \text{v}[1]$.
- Corresponde aos elementos processados, o ultimo processado é sempre o $i - 1$

A função devolve o maior índice i tal que a soma dos primeiros i elementos do array é menor ou igual a *lim*. Se $i < N$, então adicionar $v[i]$ faria a soma ultrapassar *lim* .

3.2 I - Correção (2)

2. Defina um invariante do ciclo que permita provar a correção parcial desta função.

Resolução:

Em qualquer momento do cíclo, *sum* é a soma dos primeiros i elementos do array .

- Se $i = 0$, então $sum = 0$;
- Se $i = 2$, então $sum = v[0] + v[1]$;
- Se $i = 5$, então $sum = v[0] + ... + v[4]$.

Invariante :

$$sum = \sum_{k=0}^{i-1} v[k] \ \wedge \ sum \leq lim$$

"Sum é a soma dos primeiros i elementos do array e essa soma não ultrapassa lim"

Variante :

$$N - i$$

3.3 II - Complexidade

Considere a função $fromArray$ que constrói uma árvore binária equilibrada a partir de um array.
Apresente e resolva uma recorrência para calcular o número de acessos ao array efectuados por esta função.

```
1  ABin fromArray (int v[ ], int N){
2      ABin r = NULL; int m;
3      if (N > 0){
4          r = malloc (sizeof (struct abin));
5          m = N/2;
6          r -> esq = fromArray (v, m);
7          r -> valor = v[m];
8          r -> dir = fromArray (v + m + 1, N - m - 1);
9      }
10     return r;
11 }
```

Resolução:

- $r->\ valor = v[m]$ é o **único acesso** ao array (leitura) ;
- $r->\ dir = fromArray(v+m+1, N-m-1)$ a aritmetica de ponteiros $v+m+1 \implies v[m+1]$ não conta como acesso .

Tendo em consideração o nº de acessos ao array :

- A árvore fica dividida aproximadamente em 2 partes $\frac{N}{2}$ direita e esquerda ;
- Durante as recorrências :

$$\begin{aligned} T(N) &= T(\frac{N}{2}) + T(\frac{N}{2}) + 1 \\ &= 2T(\frac{N}{2}) + 1 \end{aligned} \tag{3.1}$$

(onde 1 corresponde ao acesso ao array (leitura))

Caso Base:

$N == 1$ (faz exatamente um acesso, as duas chamadas não acontecem)

$$T(1) = 1$$

Recorrência:

$$\begin{aligned} T(N) &= 2T(\frac{N}{2}) + 1 \\ &= 2(2T(\frac{N}{4}) + 1) + 1 \\ &= 4T(\frac{N}{4}) + 2 + 1 \\ &= 4(2T(\frac{N}{8}) + 1) + 3 \\ &= 8T(\frac{N}{8}) + 4 + 3 \\ &= 8T(\frac{N}{8}) + 7 \end{aligned} \tag{3.2}$$

- Depois de k expanssões :

$$T(N) = 2^k T(\frac{N}{2^k}) + (2^k - 1)$$

- A recurssão termina quando :

$$\frac{N}{2^k} = 1 \equiv N = 2^k \equiv k = \log_2 N$$

(Nota: $b^{\log b^x} = x$)

- Assim temos :

$$\begin{aligned} T(N) &= 2^{\log_2 N} T(\frac{N}{2^{\log_2 N}}) + (2^{\log_2 N} - 1) \\ &= N * T(1) + (N - 1) \end{aligned} \tag{3.3}$$

(Nota: $T(1) = 2 * T(0) + 1 = 1$, uma vez que $T(0) = 0$)

$$\begin{aligned} T(N) &= N * 1 + N - 1 \\ &= N + N - 1 \\ &= 2N - 1 \end{aligned} \tag{3.4}$$

Logo $\underline{\theta(N)}$.

3.4 III - Tabelas de Hash

Considere uma tabela de Hash de inteiros implesmentada com um array dinâmico com linear probing e onde as remoções são feitas marcando as respectivas células como apagadas (D); U e F são usados para marcar células ocupadas e livres.

Considere ainda que quando o rácio de células removidas atinge um determinado valor se faz uma *garbage collection* que consiste em (sem qualquer redimensionamento) recalcular a tabela sem as células removidas (e sem redimensionar a tabela) .

Assumindo que a função que a função de hash é definida apenas como $hash(x) = x \,\%\, size$, diga qual o estado da seguinte tabela após a referida opreação de *garbage collection*.

D	D	U	U	F	F	F	F	D	D	U
99	45	23	76	33	22	11	55	30	129	87
0	1	2	3	4	5	6	7	8	9	10

size = 11

Resolução:

Tendo em consideração ($hash(x) = x\%size$) as novas posições serão:

- $23 = 11 * 2 + \mathbf{1}$, posição 1
- $76 = 11 * 6 + \mathbf{10}$, posição 10
- $87 = 11 * 7 + \mathbf{10}$, posição 10

U	U	F	F	F	F	F	F	F	F	U
87	23									76
0	1	2	3	4	5	6	7	8	9	10

3.5 IV - Análise amortizada (1)

Relativamente à tabela referida na questão anterior, assuma que a operação de *garbage collection* é realizada quando pelo menos 1/4 das células estão apagadas .

Assuma ainda que o custo real de inserção e remoção de um elemento são feitos em tempo constante (1).

1. Qual o custo real da operação de *garbage collection* definida acima.

Resolução:

Garbage Collection :

- percore tabela ;
- ignora os D ;
- reinsere todos U numa nova tabela .

Custo real :

- custo de inserção e remoção = 1 ;
- Garabge Collection não remove explicitamente, simplesmente os **D's** desaparecem quando criamos a nova tabela e copiamos os elementos U .

O custo vai ser de :

$$\text{percorrer a tabela toda } (size = 11)$$

$$+$$

$$\text{reinserir 3 elementos U}$$

Logo o custo real $\implies 11 + 3 = 14$ ou de forma assintótica $\theta(n)$

3.6 IV - Análise amortizada (2)

2. Mostre que os custos amortizados das operações de remoção e garbage collection são constantes .

Resolução:

O garbage collection acontece quando :

$$numero\ de\ D's \geq \frac{size}{4}$$

Ou seja, numa tabela de tamanho n, só acontece quando existe pelo menos $\frac{n}{4}$ remoções.

Custo da garbage collection $\implies O(n)$

Para mostar que o custo amortizado é constante, temos de ter em consideração que :

- O custo do garbage collection vai ser dividido pelas romoções que aconteceram antes ;
- Para acontecer garbage collection teve de exister pelo menos $\frac{n}{4}$ remoções ;
- Cada remoção pode "guardar" um crédito .

Se cada remoção guarda 4 creditos, então no momento do garbage collection temos :

$$4 * \frac{n}{4} = n \ \ creditos$$

Esses créditos pagam todo o custo do garbage collection .

Vamos analisar agora o **custo real**:

- **Remoção**

  $1 + 4 = 5$, (onde 4 é os créditos guardados) logo o custo armotizado é uma constante.

- **Garbage Collection**

  O custo real é n, mas esse custo já foi pago pelos créditos acumulados nas remoções .

Logo o custo amortizado é 0, ou seja constante. Como:

- cada remoção tem um custo amortizado constante ;
- o garbage collection é pago pelos créditos acumulados .

O custo amortizado de **ambas** as operações é $\theta(1)$.

Resposta formal :

O garbage collection só occore após existirem pelo menos $\frac{n}{4}$ células apagadas. Se cada remoção guarda 4 unidades de crédito, então, quando o garbage collection ocorre, existem $4 * (\frac{n}{4}) \equiv n$ créditos acumulados, suficientes para pagar o seu custo $O(n)$. Assim, cada remoção tem custo amortizado de $(1 + 4 = 5)$ e o garbage collection fica totalmente pago pelos créditos acumulados. Logo, ambas as operações têm custo amortizado $\theta(1)$.

3.7 V - Grafos

Considere uma matriz de inteiros que representa as altitudes dos pontos de um mapa. Considere ainda um robot que se pode deslocar nesse mapa segundo os habituais movimentos Norte, Sul, Este e Oeste. Defina uma função $int\ \ maior_descida\ \ (int\ \ mapa[N][N])$ que calcula o comprimento do maior caminho no mapa que só implica descidas (todos os movimentos correspondem a passar para um ponto com menor altitude). Comece por descrever sucintamente a estratégia que irá usar para resolver este problema.
Por exemplo, para a matriz abaixo, a função deve retornar 10, correspondendo às posições marcadas .

```
int alturas[10][10] = {{10, 4, 3, #10, 1, 7, 5, 5, 4,1}
                      ,{ 1, 2, 5, #9, 2, 5, 4, 6, 6,1}
                      ,{ 9, 3,10, #8, 9, 3 , 4,10, 2,1}
                      ,{ 6, 5, #6, #7, 5, 6, 6, 4, 6,1}
                      ,{ 7, 8, #5,10, 1,12, 1, 1, 7,1}
                      ,{ 4,15, #4, #3, #2, 8, 5, 6,11,1}
                      ,{ 2, 3,10, 8, #1, 4, 3, 3, 9,1}
                      ,{10,12, 4,10, #0, 7, 2, 4, 2,1}
                      ,{ 2, 1,10, 3, 1, 6, 8,13, 8,1}
                      ,{ 5,10, 5, 8,13, 4, 5,15, 1,1}
                      };
```

Resolução:

Utiliza-se uma **DFS com memorização**. Para cada posição da matriz calcula se o comprimento do maior caminho descendente que começa nessa posição. Esse valor é guardado na matriz *memo*, avitando recalcular o mesmo subproblema. No final percorrem-se todas as posições da matriz e devolve-se o maior comprimento encontrado.

```
int dfs( int mapa [N][N], int memo[N][N], int i, int j){
    if (memo[i][j] != 0)
        return memo[i][j];
        //Significa que ja calculei o maior caminho desta posicao.

    int melhor = 1;
    //independetemente de sair do sitio, onde esta ja conta como 1

    //Norte
    if(i > 0 && mapa[i-1][j] < mapa[i][j]){
        int x = 1 + dfs(mapa, memo, i-1, j);
        if (x > melhor)
            melhor = x;
    }
    //Sul
    if(i < N - 1 && mapa[i+1][j] < mapa[i][j]){
        int x = 1 + dfs(mapa, memo, i+1, j);
        if (x > melhor)
            melhor = x;
    }
    //Este
    if(j < N - 1 && mapa[i][j+1] < mapa[i][j]){
        int x = 1 + dfs(mapa, memo, i, j+1);
        if (x > melhor)
            melhor = x;
    }
    //Oeste
    if(j > 0 && mapa[i][j-1] < mapa[i][j]){
        int x = 1 + dfs(mapa, memo, i, j-1);
        if (x > melhor)
            melhor = x;
    }
    memo[i][j] = melhor;
```

```
34      //vai calcular a distancia de cada vertice na
35      //procura e gravar no memo, entao quando entra
36      //em recursao vai ate ao fundo onde marca o
37      //ultimo com 1 e ao voltar vai somando, por isso
38      //1 -> 2 -> 3, ate chegar ao primeiro
39      //faz isso para toas as possibilidades e devolve o maior
40
41      return melhor;
42  }
43
44  int maior_descida( int mapa[N][N]){
45
46      int memo[N][N] = {0};
47      int m_altura = 0;
48
49      for (int i = 0; i < N; i ++){
50          for (int j = 0; j < N; j ++){
51              int x = dfs( mapa, memo, i, j);
52
53              if(x > m_altura)
54                  m_altura = x;
55          }
56      }
57      return m_altura;
58  }
```

Nota : *memo* não precisa de retornar pois passamos ponteiros, logo guarda sempre os próximos.

Capítulo 4

Teste 1 - 22/23

4.1 I - Especificação

Considere a seguinte função que determina o primeiro índice onde dois arrays diferem. Se os arrays forem iguais, a função devolve N (o tamanho dos arrays). Apresente uma especificação para este problema.

```
1 int difInd (int a[], int b[], int N) {
2     // Pre: ?
3     int i = 0;
4     i = 0;
5     while (i < N && a[i] == b[i])
6         i = i + 1;
7     // Pos: ?
8     return i;
9 }
```

Resolução:

Pré :

$$N \geq 0 \ \wedge \ a[0...N-1] \ \ e \ \ b[0...N-1] \text{ são arrays válidos}$$

Pós :

$$(i = N \ \wedge \ \forall_{0 \leq k < N},\, a[k] = b[k]) \ \vee \ (0 \leq i < N \ \wedge \ a[i] \neq b[i] \ \wedge \ \forall_{0 \leq k < i}\, , a[k] = b[k])$$

- $(0 \leq i < N \ \wedge \ a[i] \neq b[i])$, corresponde ao i onde parou ;
- $\forall_{0 \leq k < i}\, , a[k] = b[k])$, esta parte garante que até parar, tudo anterior estava certo .

4.2 II - Correção

Indique qual o invariante e variante que permitem verificar a correção total da seguinte função que, dado um array com os pesos de N produtos que se pretende comprar num supermercado, e a capacidade C dos sacos desse supermercado, determina o número de sacos necessários para transportar todos os produtos. Por exemplo, se os pesos dos produtos forem $\{3, 6, 2, 1, 5, 7\}$ e $C == 10$ a função devolve 3.

```
int sacos(int p[], int N, int C) {

    // Pre: N > 0 && C > 0 && forall_{0 <= k < N} 0 < p[k] <= C
    int r = 1, t = p[0], i = 1, j = 0;
    while (i < N) {
        // Inv: ?
        // Var: ?
        if (t + p[i] > C) {
            j = i; t = 0; r = r + 1;
        }
        t = t + p[i]; i = i + 1;
    }
    return r;
    // Pos: 0 < r <= N && r*C >= sum_{0 <= k < N} p[k]
}
```

Resolução:

Nota :

- $\forall_{0<=k<N},\ 0 < p[k] \leq C$, significa que nenhum produto vale mais que a capacidade do saco ;
- i $->$ proximo produto a processar ;
- j $->$ primeiro produto do saco atual ;
- t $->$ peso do saco atual ;
- r $->$ nº sacos usados .

Variante :

$$N - 1$$

Invariante :

$$t = \sum_{k=j}^{i-1} p[k] \ \wedge \ 0 < t \leq c$$

- Apesar de $t = 0$ a meio do ciclo, isso nunca chega a ser verificado, logo $t > 0$;
- t é o peso total dos produtos colocados no saco atual (de j a $i-1$) e esse peso nunca excede a capacidade do saco ;
- Podia também ser acrescentado $1 \leq r < i$ porque nunca existe mais sacos do que produtos processados .

4.3 III - Complexidade algoritmos iterativos

Analise a complexidade da função sacos (conte o número de operações aritméticas), identificando o pior e melhor caso.

Resolução:

Melhor caso : Todos os produtos cabem no mesmo saco, pelo que o *if* nunca é executado. Em cada um das *N-1* iterações, são realizadas 3 operações aritméticas ($t+p[i]$, $t = t+p[i]$, $i = i+1$), totalizando $3(N-1)$ operações.

Pior caso : Cada produto obriga à criação de um novo saco, pelo que *if* é sempre executado. Em cada uma dos $N-1$ iterações são realizadas 4 operações aritméticas ($t+p[i]$, $r = r+1$, $t = t+p[i]$, $i = i+1$), totalizando $4(N-1)$ operações.

Em ambos os casos, a complexidade assintótica é $\theta(N)$.

4.4 IV - Complexidade algoritmos recursivos

Considere agora a seguinte função que calcula o maior número de produtos que se pode comprar no supermercado com um único saco de capacidade C. Por exemplo, se os pesos dos produtos forem $\{3, 6, 2, 1, 5, 7\}$ e $C == 12$ a função devolve 4, correspondente a comprar o 1º, 3º, 4º e 5º produtos.

```
1  int ensaca(int p[], int N, int C) {
2
3      int r,t;
4      if (N == 0) return 0;
5      r = ensaca(p + 1, N - 1, C);
6      if (p[0] <= C) {
7          t = 1 + ensaca(p + 1, N - 1, C - p[0]);
8          if (t > r) r = t;
9      }
10     return r;
11 }
```

Analise a complexidade da função ensaca (conte o número de acessos ao array), identificando o pior e melhor caso.

Resolução:

Vamos começar com um exemplo da função a ser executada para entender melhor :

$P = \{3, 4, 2\}$
$N = 3$
$C = 5$

Qual é o maior nº de produtos que conseguimos levar ?

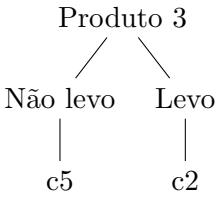
1º chamada ;

$$ensaca(\{3, 6, 2\}, 3, 5)$$

- $r = ensaca(\{6, 2\}, 2, 5) \implies$ "E se eu não levar o primeiro produto ?"
- $if(3 \leq 5) \implies$ (verdade)

- $t = 1(\textit{produto adicionado}) + \textit{ensaca}(\{6,2\},2,2) \implies$ "levo o produto de peso 3 e ainda sobra capacidade"

De uma forma visual,



Agora teremos duas partes para resolver,

1. $\textit{ensaca}(\{6,2\},2,5)$
2. $1 + \textit{ensaca}(\{6,2\},2,2)$

Comecemos pelo **1**:

```
ensaca({6,2},2,5){ // isto vai ser 1 por conta de **(2)
    ...
    r = ensava({2}, 1, 5) // representa nao levar 6
    if(6 <= 5)//Falso , avanca
}
```

```
ensaca({2},1,5){
    ...
    r = ensava({}, 0, 5) // isto vai ser = 0
    if(2 <= 5){//Verdade
        t = 1 + ensaca({}, 0, 3) // o ensaca vai ser = 0 logo t = 1
        if( 1 > 0) r = 1;
    }
    return 1;
}
```

Logo,

```
ensaca({6,2},2,5){
    ...
    r = ensava({2}, 1, 5) // vai ser = 1
    if(6 <= 5)//Falso
        ...
    return r = 1;
}
```

Agora o lado **2**:

```
ensaca({6,2},2,2){
    ...
    r = ensava({2}, 1, 2)
    if(6 <= 5){//Falso, avanca
        ...
```

```
    }
    return r;
}
```

```
ensaca({2},1,2){
    ...
    r = ensava({}, 0, 2) // isto vai ser = 0
    if(2 <= 5){//Verdade
        t = 1 + ensaca({}, 0, 0) // o ensaca vai ser = 0 logo t = 1
        if( 1 > 0) r = 1;
    }
    return 1;
}
```

Logo,

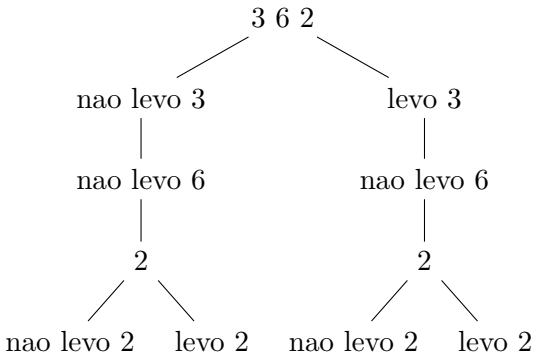
```
ensaca({6,2},2,2){
    ...
    r = ensava({2}, 1, 2) // vai ser = 1
    if(6 <= 5)//Falso
        ...
    return r = 1;
}
```

Assim, voltando a **2**, temos que $1 + ensaca(\{6,2\},2,2) \implies 1+1=2$.

Voltamos à primeira chamada, ficamos com :

```
ensaca({3,6,2},3,5){
    ...
    r = ensava({6,2}, 2, 5) // = 1
    if(3 <= 5){
        t = 1 + ensaca({6,2}, 2, 2) // = 2
        if( 2 > 1) r = 2;
    }
    return r = 2;
}
```

Ou seja, para cada produto temos :



Cada produto gera 2 escolhas .

É um programa lento pois para k produtos temos 2^k possibilidades.

Tendo em consideração que o exercício pede para contas os acessos ao array :

- Ao logo deste proframa acedemos ao array sempre que temos $p[0]$, logo em $if(p[0] \leq c$ e em $ensaca(p+1, N-1, c-p[0])$;
- Portanto, numa chamada onde o produto cabe, fazemos **2 acessos ao array** .

Pior caso : Todos os produtos cabem.

Cada chamada gera 2 chamadas recursivas, como neste caso acedemos ao array **2 vezes** por chamda, temos:

Caso Base : $T(0) = 0$

Caso recorrência :

$$\begin{aligned} T(N) &= T(N-1) + T(N-1) + 2 \\ &= 2T(N-1) + 2 \\ &= 2(2T(N-2) + 2) \\ &= 4T(N-2) + 4 \end{aligned} \tag{4.1}$$

Para a recorrencia terminar, uma vez que $N-2$ vai aumentando, podemos considerar que a cada chamada $N-k$, logo

$$2^k T(N-k) + 2^k$$

Assim para T terminar $N - k = 0 \equiv N = k$

$$\begin{aligned} T(N) &= 2^N T(N-N) + 2^N \\ &= 2^N * T(0) + 2^N \\ &= 2^N \end{aligned} \tag{4.2}$$

Assim, $\theta(2^N)$.

Melhor caso : Nenhum produto cabe, logo nunca entra no if e só é feito 1 acesso de comparação e uma chamada recursiva.

$$\begin{aligned} T(N) &= T(N-1) + 1 \\ &= 2 * (T(N-2) + 1) \\ &= 2T(N-2) + 2 \end{aligned} \tag{4.3}$$

Como a cada chamada aumenta +1, então seja $k = 2$,

$$kT(N-k) + k$$

Para $T(N-k)$ terminar $N - k = 0 \equiv N = k$,

$$\begin{aligned} T(N) &= N * T(N-N) + N \\ &= N * T(0) + N \\ &= N \end{aligned} \tag{4.4}$$

Logo $\theta(N)$.

Capítulo 5

Teste 2 - 22/23

5.1 I - Árvores AVL

Defina uma função AVLTree build (int a[], int N) que constrói uma árvore AVL a partir de um array ordenado. Garanta que os factores de balanço ficam bem preenchidos e que a função executa em tempo linear.

```c
typedef struct avlnode {
    int valor; int bal;
    struct avlnode *esq, *dir;
} *AVLTree;
```

Resolução:

```c
AVLTree buildAux(int a[ ], int N, int *h){
    //quando chega ao fim e comeca a contar a altura a partir do fim
    //(importante ser *h se nao perdemos o valor)
    if(N == 0){
        *h = 0;
        return Null;
    }
    int m = N/2;
    AVLTree r = malloc(sizeof(struct avlnode));
    int he, hd;
    r -> esq = buildAux(a, m, &he);
    r -> valor = a[m];
    r -> dir = buildAux(a + m + 1, N - m - 1, &hd);
    r -> bal = hd - he;
    // se he > d -> *h = he +1; se nao *h = hd +1;
    *h = 1 + (he > hd ? he : hd);
    return r;

}
// a altura e calculada na subida da recursao

AVLTree build(int a[ ], int N){
    int h;
```

```
24      return buildAux(a, N, &h);
25  }
```

Cada chamada devolve duas coisas :

- a árvore e a altura dessa árvore

Assim, cada nó é visitado uma única vez, logo :

$$T(N) = T(\frac{N}{2}) + T(\frac{N}{2}) + O(1)$$

Logo,

Caso Base :

$$T(0) = 0, \text{ não é visitado nenhum nó}$$

Caso Recursivo :

$$\begin{aligned} T(N) &= 2T(\frac{N}{2}) + 1 \\ &= 2(2T(\frac{N}{4}) + 1) + 1 \\ &= 4T(\frac{N}{4}) + 3 \end{aligned} \tag{5.1}$$

Após kexpansões :

$$2^k T(\frac{N}{2^k}) + (2^k - 1)$$

Para $\frac{N}{2^k}$ terminar, $\frac{N}{2^k} = 1 \equiv N = 2^k \equiv k = \log_2 N$

$$\begin{aligned} T(N) &= N * T(1) + (N - 1) \\ &(T(1) = 2 * T(0) + 1 = 1) \\ &= N * 1 + (N - 1) \\ &= N + N - 1 \\ &= 2N - 1 \end{aligned} \tag{5.2}$$

Logo $\theta(N)$, a função tem tempo linear .

5.2 II - Análise de caso médio

Faça a análise de caso médio da função f, assumindo que a é um array de $0s$ e $1s$ e que g é uma função da classe $\theta(2^N)$. Comece por identificar o melhor e o pior casos.

```
1  int f(int a[], int N) {
2      int b = 0;
3      for (int i = 0; i < N; i++)
4          if (a[i] == 1) b++;
5      if (b == 1) return g(a,N);
6      else return 0;
7  }
```

Resolução:

O ciclo conta quantos 1s existem no array. No fim :

- se existir exatamente $1 ->$ chama $g(a, N)$;
- caso contrário $->$ devolve 0 .

Melhor caso :

Mesmo no melhor caso, o ciclo faz N iterações

- se todos são 0's ;
- ou dois ou mais 1's ;
- g não é chamada .

Logo $\theta(N)$.

Pior caso :

Acontece quando existe exatamente um 1 no array

- o ciclo faz $\theta(N)$;
- depois chama $g(a, N)$, com custo $\theta(2^N)$.

Logo $\theta(N + 2^N) = \theta(2^N)$

Caso médio :

Assumindo que cada posição do array é **0** ou **1** com igual probabilidade $(\frac{1}{2})$, qual é a probabilidade de existir exatamente 1 ?

- Nº de arrays com exatamente 1
  - escolheremos uma posição $->$ N possibilidades
- Nº total de array binários
  - 2^N
- Probabilidade $P(b = 1) = \frac{N}{2^N}$

O **tempo médio esperado** :

Custo Esperado (E)

$$\begin{aligned} E(T) &= \theta(N) + P(b = 1) * \theta(2^N) \\ E(T) &= \theta(N) + (\frac{N}{2^N}) * \theta(2^N) \\ &= \theta(N) + \theta(N) \\ &= \theta(N) \end{aligned} \tag{5.3}$$

Assim o caso médio $\theta(N)$.

- A função exponêncial só é chamada quando há exatamente um 1 no array ;
- Mas esse evento tem probabilidade $\frac{N}{2^N}$, que é muito pequena. Por isso, apesar de existir um pior caso exponêncial, o custo médio continua linear .

5.3 III - Min-heaps

Implemente uma função int decrease(int x, int y, int h[], int N) que dada uma min-heap h com N elementos e um valor x que está na min-heap, troque esse valor pelo valor y, assumindo que y ¡ x. Analise o pior e melhor caso da função implementada.

Resolução:

Notas importantes :

- Mini-heap $->$ $pai \leq filhos$;
- A mini-heap não está organizada como uma BST, logo temos de procurar em todo o array .

```
1  int decrease (int x, int y, int h[ ], int N){
2
3      int i;
4      //primeiro achar e substituir
5      for(i = 0; i < N && h[i] != x; i++);
6      //percorre o array ate achar o indice onde x esta
7
8      if(i == N)
9          //nao existe o x
10         return 0;
11
12     h[i] = y;
13
14     //fazer bubble up se o filho for menor que o pai
15     //indice do pai (i-1)/2, indice do irmao i-1
16     while (i > 0 && h[(i-1)/2] > h[i]){
17
18         int aux = h[i];
19         h[i] = h[(i-1)/2];
20         h[(i-1)/2] = aux;
21
22         i=(i-1)/2;
23     }
24     return i;
25 }
```

A complexidade tem duas fases :

Pior caso :

- Procurar elemento ;
  - percorre todo o array $\theta(N)$.
- Bubble up ;
  - no pior caso sobe até à raiz ;
  - a altura da heap é $\log N$, logo $\theta(\log N)$.

Complexidade Total $-> \theta(N) + \theta(\log N)$, como $N > \log N$, então $\underline{\theta(N)}$.

Melhor caso :

- x está logo em $h[0]$;
- a procura demora apenas 1 posição e o ciclo while não executa .

Logo, $\underline{\theta(1)}$.

5.4 IV -Grafos

Implemente a função int maior(int image[N][N]) que dado um bitmap com uma imagem de dimensão N por N, devolva o tamanho da maior figura (pixeis contíguos horizontal ou verticalmente) nela contida. Por exemplo, no seguinte bitmap a maior figura tem tamanho 7. Pode assumir que os píxeis a 1 nunca têm coordenadas 0 ou N-1.

```
{{0,0,0,0,0,0},
{0,1,0,1,1,0},
{0,1,1,1,0,0},
{0,1,0,0,1,0},
{0,0,1,1,1,0},
{0,0,0,0,0,0}}
```

Resolução:

Ideia geral :

- usar DFS ;
- usar apenas uma matriz $visitados[N][N]$;
- quando visito um pixel :
  - marca como visitado ;
  - contar 1's ;
  - explorar os quatro vizinhos ;
  - somar os resultados .

```
int dfs( int img[N][N], int vis[N][N], int i, int j){

     if(i < 0 || i >= N || j < 0 || j >= N)
        return 0;

    if(img[i][j] == 0)
        return 0;

    if(vis[i][j]) // se ja foi visitado
        return o;

```

```
12      vis[i][j] = 1;
13
14      return 1
15             + dfs(img, vis, i-1, j)
16             + dfs(img, vis, i+1, j)
17             + dfs(img, vis, i, j-1)
18             + dfs(img, vis, i, j+1);
19  }
20
21  int maior( int img[N][N]){
22
23      int vis[N][N] = {0};
24      int melhor = 0;
25
26      for(int i = 0; i < N; i++)
27          for( int j = 0; j < N; j++)
28              if(img[i][j] == 1 && !vis[i][j]){
29
30                  int tam = dfs(img, vis, i, j);
31                  if(tam > melhor)
32                      melhor = tam;
33              }
34
35      return melhor;
36  }
```

Cada pixel é visitado **1 vez** logo, $\theta(N^2)$.

- É o melhor possivel, pois temos de olhar para a imagem toda.

Capítulo 6

Teste 1 - 23/24

6.1 I - Especificação

Considere a seguinte função que, dado um número inteiro x entre 1 e 9, e um vector de números com valor igualmente entre 1 e 9, procura a primeira posição do vector que contém o valor 10-x. Escreva uma pré-condição e uma pós-condição que descrevam este comportamento.
Observe que a pós-condição deve referir duas situações distintas, correspondentes às duas situações em que o ciclo pode terminar.

```
1 int addsTo10 (int x, int a[], int N) {
2     // Pre: . . .
3     int i = 0 ;
4     while (i<N && a[i]+x != 10)
5         i=i+1 ;
6     // Pos: (i==N && . . .) || (i<N && . . .)
7     return i ;
8 }
```

Resolução:

Pré :

$$N \geq 0 \ \wedge \ 1 \leq x \leq 9 \ \wedge \ a[0...N-1]\ valido \ \wedge \ \forall_{0 \leq k < N},\ 1 \leq a[k] \leq 9$$

Pós :

$$(i = N \ \wedge \ \forall_{0 \leq k < N},\ a[k] + x \neq 10) \ \vee \ (i < N \ \wedge \ a[i] + x = 10 \ \wedge \ \forall_{0 \leq k < i},\ a[k] + x \neq 10)$$

Sempre que :

- primeiro elementeo ;
- primeiro índice ;
- primeira ocorrência .

Nunca esquecer de acrescentar $\forall_{k<i}$

"Todos os elementos anteriores não satisfazem a condição ."

6.2 II - Correção

Escreva um invariante para a função da Questão 1. Deverá incluir o seguinte:

1. Informação sobre os valores possíveis da variável i ;
2. Informação sobre a relação entre os valores contidos nas posições já visitadas do array e o valor da variável x ;

Resolução:

Relativamente a **1** : Informação sobre os vetores possíveis de i...

$$0 \leq i \leq N$$

Pois quando termina o ciclo, i pode valor N.

Relativamente a **2** : Informação sobre as posições visitadas...

Sabemos que todos vizitados não satisfazem a condição da paragem logo

$$\forall_{0 \leq k < i},\ a[k] + x \neq 10$$

Logo **Invariante** : $0 \leq i \leq N\ \wedge \forall_{0 \leq k < i},\ a[k] + x \neq 10$

Variante : $N - i$

6.3 III - Complexidade algoritmos iterativos

Analise o melhor caso, pior caso, e caso médio da complexidade da função addsTo10, contando o número de comparações (a[i]+x != 10). Para o caso médio apresente a solução usando somatórios.

Resolução:

Nota :Contando o nº de comparações $(a[i] + x \neq 10)$;

Melhor caso

- o valor procurado está logo em $a[o] \implies$ 1 comparação $\implies \theta(1)$

Pior caso

- o valor está na última posição ;
- o valor não existe ;

- em ambos são feitos N comparações $\implies$ $\theta(N)$

Caso Médio

- supunhamos que o primeiro elementeo que satisfaz $a[i] + x == 10$ está na posição i $\implies$ a função faz $i + 1$ comparações ;
- se assumimos que cada posição é igualmente provavel :

$$P(i) = \frac{1}{N},\ para\ i = 0, ..., N - 1$$

- Então o nº médio de comparações é :

$$E[c] = \sum_{i=0}^{N-1} \frac{1}{N}(i + 1)$$

Extra : Caso o enunciado disser que o elemento pode não existir .

Consideremos $N + 1$ casos possiveis :

- encontrar
- não encontrar

Imaginemos para um caso de $N = 4$

Valor na posição..	Comparações
0	1
1	2
2	3
3	4
O valor não existe	4

Temos 4 posições possiveis de encontrar +1 caso em que ele não existe, logo $N + 1$ caos.

Se assumirmos que todos esses casos são igualmente provaveis, cada um tem probabilidade $\frac{1}{N+1}$.
Logo esperança :

$$E[c] = \frac{1}{N + 1}(1 + 2 + 3 + 4...N + N)$$

Assim,

$$E[c] = \frac{1}{N + 1} * (\sum_{i=0}^{N-1} (i + 1) + N)$$

(onde N representa o cado de o valor não existir)

6.4 IV - Complexidade de algoritmos recursivos

Considere agora a seguinte função que decide se um vector de inteiros, com valores entre 1 e 9, contém ou não um par de elementos cuja soma vale 10.

```
1 int par10Rec (int u[], int N) {
2     int p ;
3     if (N<2) return 0 ;
4     p = addsTo10(u[0], u+1, N-1) ;
5     if (p<N-1) return 1 ;
6     return (par10Rec (u+1, N-1)) ;
7 }
```

De forma a analisar a complexidade desta função, identifique o melhor e pior casos e, para o pior caso, apresente e resolva a recorrência que traduz essa complexidade.

Resolução:

p = addsTo10(u[0], u+1, N-1) ;

- procura nos $N-1$ elementos seguintes um valor que some 10 com $u[0]$;
- se encontar (*if (p¡N-1) return 1 ;*) termina ;
- se não encontar, chama recursivamente (*return (par10Rec (u+1, N-1)) ;*), ou seja, elimina o primeiro elemento e tenta de novo .

Melhor Caso :

Acontece qunado o 1º elemento já forma um par com o segundo. **Ex:.**$[3, 7, 5, 1]$

- *addsto10* faz apenas 1 comparação ;
- a funçao termina imediatamente .

Logo $\theta(1)$.

Pior caso :

Quando não existe nenhum par cuja soma seja 10.
(**Nota :**não é quando o par está no fim, porque eventualmente ele será encontrado e a recursão termina).

Se nenhum par existir :

- 1ª chamda $->$ *addsto10* percore $N-1$;
- 2ª chamada $->$ $N-2$;
- 3ª chamada $->$ $N-3$;
- ...
- até não ter mais pares .

Na recorrência :

- Uma chamada a *addsto10* sobre um vetor $N-1$, que no pior caso custa $N-1$ comparações .

Logo

$$T(N) = T(N-1) + (N-1) + O(1)$$

(onde $O(1)$, corresponde às comparações do *if*)

Caso Base :

$$T(1) = 0$$

Parte Recursiva :

$$\begin{aligned} T(N) &= T(N-1) + (N-1) \\ &\quad T(N-2) + (N-2) + (N-1) \\ &\quad = T(N-3) + (N-3) + (N-2) + (N-1) \end{aligned} \tag{6.1}$$

Após k expansões :

$$T(N) = T(N-k) + \sum_{i=N-k}^{N-1} i$$

Quando $N-k=1$ temos $k=N-1$, Logo

$$T(N) = T(1) + \sum_{i=1}^{N-1} i$$

Sabemos que

$$\sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

Assim, temos que :

$$T(N) = \frac{N(N-1)}{2} = \theta(N^2)$$

Capítulo 7

Teste 2 - 23/24

7.1 I - Tabelas de Hash

Considere uma tabela de números inteiros implementada com endereçamento aberto e linear probing, sobre um array de comprimento 11, com a função $hash(x) = x\%11$, e com reaproveitamento de posições entretanto libertadas. Simule a evolução do seu estado a partir da tabela inicialmente vazia, quando executa a seguinte sequência de operações:

Insert 8 ;
Insert 18 ;
Insert 7 ;
Insert 10 ;
Remove 8 ;
Insert 29 ;
Insert 20 ;

Deve desenhar a tabela em todos os passos da sequência acima

Resolução:

Tendo em consideração a funçao $hash(x) = x\%11$:

Insert 8 :

$$8 = 11 * 0 + 8$$

0	1	2	3	4	5	6	7	8	9	10
								8		

Insert 18 :

$$18 = 11 * 1 + 7$$

0	1	2	3	4	5	6	7	8	9	10
							18	8		

Insert 7 :

$$7 = 11 * 0 + 7$$

A posição 7 está ocupada, então percorremos até à posição free

0	1	2	3	4	5	6	7	8	9	10
							18	8	7	

Insert 10 :

$$10 = 11 * 0 + 10$$

0	1	2	3	4	5	6	7	8	9	10
							18	8	7	10

Remove 8 :

0	1	2	3	4	5	6	7	8	9	10
							18	D(delete)	7	10

Insert 29 :

$$29 = 11 * 2 + 7$$

A posição 7 está ocupada, logo seguimos para a livre mais próxima, neste caso 8 está livre

0	1	2	3	4	5	6	7	8	9	10
							18	29	7	10

Insert 20 :

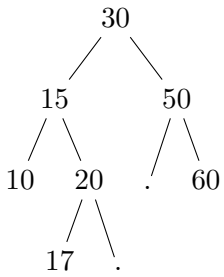
$$20 = 11 * 1 + 9$$

A posição 9 está ocupado, logo seguimos para a livre mais próxima

0	1	2	3	4	5	6	7	8	9	10
20							18	29	7	10

7.2 II - Árvores Binárias / AVL

Recorde que as estruturas de dados conhecidas como heaps são árvores binárias implementadas fisicamente sobre arrays. Embora não habitual, é possível implementar quaisquer outras árvores binárias sobre arrays, bastando para isso utilizar um valor especial para representar as lacunas, i.e. posições da árvore em que não se encontra qualquer nó. Por exemplo a seguinte árvore de números inteiros



pode ser representada por arrays preenchidos desta forma :

| 30 | 15 | 50 | 10 | 20 | H | 60 | H | H | 17 | ... H ... |

1. Escreva uma função checkBST (int t [], int N) que recebe um array t de comprimento N e verifica (devolvendo 0 ou 1) se representa ou não uma árvore binária de procura de números inteiros. Considere que a constante H é utilizada para representar lacunas.
2. Escreva uma função checkAVL (int t [], int N) que verifica se t representa ou não uma árvore balanceada (de acordo com o critério das árvores AVL). Analise em termos assintóticos o tempo de execução desta função.

Serão valorizadas as soluções mais eficientes.

Resolução:

Para responder a 1. :

```
1  int checkAux(int t[], int N, int i, int min, int max){
2
3      if( i >= N || t[i] == H)
4          return 1;
5
6      if(min != -1 && t[i] <= t[min])
7          return 0;
8
9      if(max != -1 && t[i] >= t[max])
10         return 0;
11
12     return checkAux(t, N, 2*i+1, min, i) &&
13            checkAux(t, N, 2*i+2, i, max);
14 }
15
16 int checkBST(int t[], int N){
```

```
17      return checkAux(t, N, 0, -1, -1);
18  }
```

Vamos imaginar uma iteração em que a cada nó faz a pergunta :

"A minha subárvore é uma BST?"

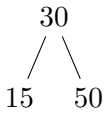
- Se sim $->$ return 1
- Se não $->$ return 0

Importante não esquecer que :

$$sim \;\&\&\; sim \implies sim$$

$$sim \; ou \; nao \;\&\&\; nao \implies nao$$

Pequena demonstração :



checkAux(30)

- *return (1.)checkAux(15) && (2.)checkAux(50);*
  1. *return checkAux(Null) && checkAux(Null)* $\implies$ *return 1 && 1 = 1;*
  2. *return checkAux(Null) && checkAux(Null)* $\implies$ *return 1 && 1 = 1;*
- logo *return checkAux(15) && checkAux(50);* $\implies$ *return 1 && 1 = 1;*

Assim , *checkAux(30)* $\implies$ *return 1*, e por isso checkBST devolve 1 .

Para responder a 2. : **Ideia geral** : Só precisamos de verificar as diferentes alturas, ou seja alturas ≤ 1.

```
1  int checkAux(int t[ ], int N, int i, int *h, ){
2
3      if(i >= N || t[i] == H){
4          *h = 0;
5          return 1;
6      }
7      int he, hd;
8
9      if(!checkAux(t, N, 2*i+1, &he))
10         return 0;
11
12     if(!checkAux(t, N, 2*i+2, &hd))
13         return 0;
14
15     if(he - hd > 1 || hd - he > 1)
16         return 0;
```

```
17
18      *h = 1 + (he > hd ? he : hd);
19
20      return 1;
21  }
22
23  int checkAVL(int t[], int N){
24      int h;
25      return checkAux(t, N, 0, &h);
26  }
```

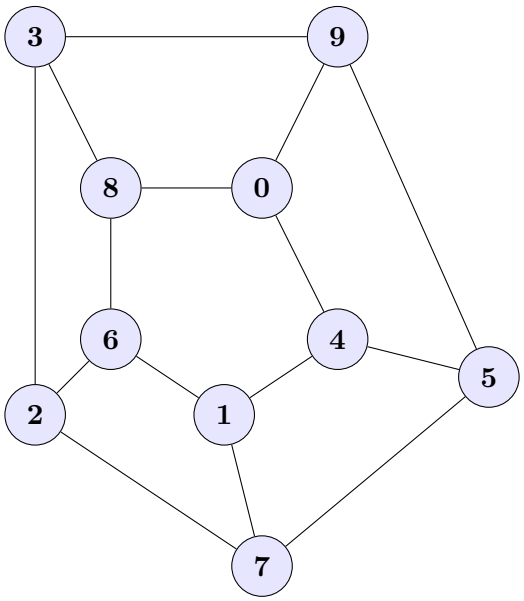
Cada nó é visistado uma vez e cada visita tem operações constantes, logo

$$T(N) = T(\frac{N}{2}) + T(\frac{N}{2}) + O(1)$$

resolvendo a recorrência (igual à recorrência do *buildAVL*), temos que $\underline{\theta(N)}$.

7.3 III - Grafos

Considere que o grafo não-orientado seguinte é representado por listas de adjacências da forma usual (i.e. cada aresta é representada por duas arestas orientadas). Considere ainda que as listas de adjacência são mantidas ordenadas por ordem crescente dos nós destino das arestas.



Desenhe:

1. A árvore de travessia em largura com origem no vértice 3
2. A árvore de travessia em profundidade com origem no vértice 1

Resolução:

Para responder a 1. :

[**3**]

[**2**, 8, 9]

[**8**, 9, 6, 7]

[**9**, 6, 7, 0]

[**6**, 7, 0, 5]

[~~**7**~~, 0, 5, 1]

[~~**0**~~, 5, 1]

[~~**5**~~, 1, 4]

[~~**1**~~, 4]

[**4**]

$$BFS = \{3, 2, 8, 9, 6, 7, 0, 5, 1, 4\}$$

Para responder a 2. :

{**1**}

{~~**4**~~, 6, 7}

{~~**0**~~, ~~1~~, 5}

{~~4~~, ~~**8**~~, 9}

{~~0~~, ~~**3**~~, 6}

{~~**2**~~, ~~8~~, 9}

{~~3~~, ~~**6**~~, 7}

{~~1~~, ~~2~~, ~~8~~}

(volta ao anteriro) {~~3~~, ~~6~~, ~~**7**~~}

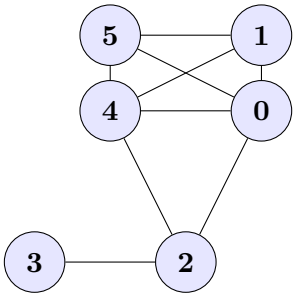
{~~1~~, ~~2~~, ~~**5**~~}

{~~4~~, ~~7~~, ~~**9**~~}

$$DFS = \{1, 4, 0, 8, 3, 2, 6, 7, 5, 9\}$$

7.4 IV - Grafos

Um "clique" de um grafo não-orientado é um conjunto de vértices que são todos adjacentes uns dos outros. No grafo em baixo, os conjuntos 0, 1, 4, 5 e 0, 2, 4 são exemplos de cliques.



Complete a seguinte definição da função *is_clique* que determina se o conjunto de vértices contidos no array v, de comprimento k, constitui um clique do grafo g (NV é o número de vértices de g).

```
struct edge { int dest; struct edge *next; };
typedef struct edge *GrafoL[MAX];
int is_clique (GrafoL g, int NV, int v[], int k) {
    ...
}
```

Analise o tempo de execução da função que escreveu. Serão valorizadas as soluções mais eficientes.

Resolução:

O que significa **clique** ?

Cada par de vértices do conjunto tem de estar ligado por uma aresta.

Ou seja, se temos $v = \{0, 4, 5\}$, temos de verificar *(0,4),(0,5),(4,5)*. Se tivermos os 3, entao é **clique**.

```
//funcao que responde "existe aresta entre a e b?"
int existeAresta(GrafoL g, int a, int b){

    for(struct edge *e = g[a]; e != NULL; e = e->next)
        if(e->dest == b)
            return 1;

    return 0;
}

//funcao principal vai garantir que verifca para todos os partes
int is_clique(GrafoL g, int NV, int v[ ], int k){

    for(int i = 0; i < k; i++)
        for(int j = i+1; j < k ; j++)
            if(!existeAresta(g, v[i], v[j]))
                return 0;
            //se nao existir, ele termina

```

```
20      return 1;
21  }
```

Complexidade :

Quantos pares de vértices verificamos ?

Se o clique tem k vértices, quando entra no ciclo

```
1  for(int i = 0; i < k; i++)
2          for(int j = i+1; j < k ; j++)
```

São executadas :

- $i = 0 \ -> $ k-1 vezes
- $i = 1 \ -> $ k-2 vezes
- $i = 2 \ -> $ k-3 vezes
- ...
- $i = k - 2 \ -> 1$ vezes
- $i = k - 1 \ -> 0$ vezes

Ou seja, o ciclo interior executa

$$(k-1) + (k-2) + ... + 1 = \frac{k(k-1)}{2} = O(k^2)$$

Custo para verificar se é par

- No pior caso percore toda a lista de adjacência ;
- Esta lista nunca pode ter mais que $NV - 1$ vértices ;

Logo

$$O(NV)$$

Assim para k^2 pares e cada um custa NV

$$O(k^2 * NV)$$

Resposta completa :

Existem aproximadamente $\frac{k(k-1)}{2}$ pares de vértices a verificar, isto é, $O(k^2)$. Para cada par é necessário percorrer a lista de adjacência de um vértices, que no pior caso pode contar até $NV - 1$ elementos. Logo tempo de execução

$$O(k^2 * NV)$$

Capítulo 8

Teste 1 - 24/25

8.1 I - Especificação

Considere a seguinte função que, dado um número inteiro x não negativo calcula a sua raiz quadrada inteira (i.e., o maior número inteiro r tal que $r * r <= x$). Por exemplo, intSqrt (25) = 5 e intSqrt (35) = 5. Complete a especificação da função fornecendo uma pré e pós condições apropriadas.

```
int intSqrt (int x) {
    //Pre: ...
    int r=0, p=1, i=1;
    while (p<=x){
        r+=1; i+=2; p+=i;
    }
    //Pos: r*r <= x && ...
    return r ;
}
```

Resolução:

Imaginemos um caso em que $x = 25$:

r	p	i
0	1	1
1	4	3
2	9	5
3	16	7
4	25	9
5	36	11

- p corresponde aos quadrados prefeitos ;
- r é a raiz quadrada inteira ;
- $p = (r + 1)^2$;
- $i = 2 * r + 1$.

Pré :

$$x \geq 0$$

Pós :

- No final queremos que r seja a raiz quadrada inteira ;
- Sabemos que $r * r \leq x$, falta garantir que o próximo quadrado ultrapasse (garantir o limite), logo $(r+1)^2 > x$

$$r * r \leq x \ \wedge\wedge \ (r+1) * (r+1) > x$$

8.2 II - Correção

Escreva um invariante e um variante que permitam provar a correcção total da função da Questão 1.

Resolução:

Invariante :
Sabemos que :

- $p = (r+1)^2$;
- $i = 2 * r + 1$;
- $r * r \leq x$.

logo **Invariante** :

$$r * r \leq x \ \wedge\wedge \ p = (r+1) * (r+1) \ \wedge\wedge \ i = 2 * r + 1$$

Variante :

$$x - r$$

- pois é inteiro, não negativo enquanto executa e diminui ;
- se fosse $x - p$, quando o ciclo termina $p > x$ logo $x - p$ é negativo.

8.3 III - Complexidade de algoritmos recursivos

A função *int subsetsum (int x, int v[], int N)* definida abaixo testa se existe um subconjunto dos elementos do array v cuja soma seja x. Apresente e resolva uma recorrência que traduza o número de acessos ao array no pior caso (comece por identificar esse caso).

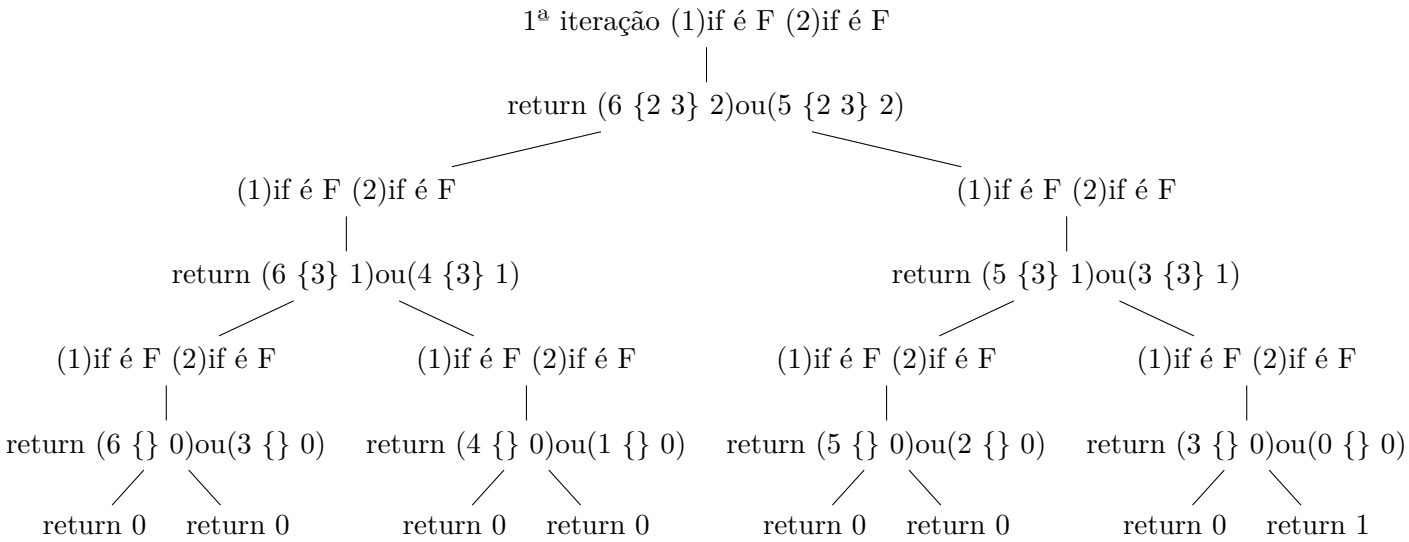
```
1 int subsetsum (int x, int v[], int N){
2     if (x==0) return 1;
3     if (N==0) return 0;
4     return (subsetsum (x, v+1, N-1) ||
5             subsetsum (x-v[0], v+1, N-1));
6 }
```

Resolução:

Nota : Quando temos *return (subsetsum (x, v+1, N-1) || subsetsum (x-v[0], v+1, N-1));*, significa que temos um "short circuit". Se a primeira chamada devolver 1, a segunda chamada nem se quer é executada.

Exemplo para visualizar :

$$v[1,2,3],\ N=3,\ x=6$$



No final *return (6 {2 3} 2) = 0 || (5 {2 3} 2) = 1*, logo **return 1**.

Melhor caso :
Quando $x == 0$ ou $N == 0$, a função termina imediatamente sem chamadas recursivas, realizando um nº constante de operações. Logo tempo é $\theta(1)$.

Pior caso :
Quando nenhum subconjunto soma x ou a solução só é encontrada na última chamada, sendo necessário explorar ambas as chamadas recursivas.
Como no enunciado diz para traudzir o nº de cacessos ao array, nesta função em cada chamada acedemos 1 vez em $v[0]$.

Caso base :

$$N == 0, \text{ (não existe acessos ao array)}$$

$$T(0) = 0$$

Recorrência :

Cada chamda faz 2 chamadas recursivas + 1 acesso ao array($v[0]$), logo

$$\begin{aligned} T(N) &= 2T(N-1)+1 \\ &= 2(2T(N-2)+1)+1 \\ &= 4T(N-2)+3 \end{aligned} \tag{8.1}$$

Apos k expansões...

$$T(N) = 2^k T(N-k) + (2^k - 1)$$

Para terminar, $N - k = 0$, assim $k = N$ (**relembrar**: caso base $T(0) = 0$)

$$\begin{aligned} T(N) &= 2^N T(0) + (2^N - 1) \\ &= 2^N - 1 \end{aligned} \tag{8.2}$$

Assim, $T(N) = \theta(2^N)$

8.4 IV - Complexidade algoritmos iterativos

A função *inc* recebe como argumento uma string de dígitos (que representa um número) e adiciona-lhe 1, devolvendo 1 se ocorreu um overflow ou 0 noutro caso. Por exemplo, se s="23499"o resultado de invocarmos inc(s,5) é 0 e s passa a conter a string "23500".

```
1  int inc (char s[], int N){
2      int i=N-1;
3
4      //enquanto encontrar um '9', muda para '0'
5      while (i>=0 && s[i] =='9')
6          s[i--] = '0';
7      if (i<0) return 1;
8
9      //quando encontra um digito entre '0' e '8' aumenta-o +1
10     s[i] = s[i] + 1;
11     return 0;
12 }
```

1. Considerando o custo desta função como o número de caracteres de s que são alterados, identifique o pior caso da execução da função e, para esse caso, indique qual o seu custo.
2. Calcule ainda o custo médio desta função.
3. Usando a função de potencial $\phi(s)$ = (soma dos dígitos de s) / 9, calcule o custo amortizado da função inc.

Resolução:

Resposta a **1.**

Pior Caso: a string é composta apenas por dígitos 9. Todos os N caracteres são alterados para '0'. Logo o custo é N.

Resposta a **2.**

Custo médio

"Quantos caracteres mudam ??" $\Longrightarrow$ Depende de quantos 9 existem no fim.
Exemplos:

- 12345
  - $5 -> 6$, custo 1
- 12349
  - $9 -> 0$, $4 -> 5$, custo 2
- 12399
  - $9 -> 0$, $9 -> 0$, $3 -> 4$, custo 3
- 12999
  - custo 4

Probabilidades :

Assumindo todos os dígitos equiprováveis, temos que :

- custo $1 ->$ último dígito não é 9

$$P = \frac{9}{10}$$

(onde 9 corresponde aos valores [0...8] e 10 aos valores [0...9])

- custo $2 ->$ último é 9 e o anterior não

$$P = \frac{1}{10} * \frac{9}{10} = \frac{9}{100} = \frac{9}{10^2}$$

(onde $\frac{1}{9}$, corresponde ao valor que é 9 e $\frac{9}{10}$ ao que não é)

- custo $3 ->$ os dois últimos são 9 e o seguinte não

$$P = \frac{1}{10} * \frac{1}{10} * \frac{9}{10} = \frac{9}{10^3}$$

Começamos a ver que

$$P(custo = k) = \frac{9}{10^k}$$

Então

$$E[c] = 1 * \frac{9}{10} + 2 * \frac{9}{10^2} + 3 * \frac{9}{10^3} + ... + k * \frac{9}{10^k}$$

Esta série converge, assim a probabilidade de altura k caracteres é $\frac{9}{10^k}$. Logo

$$E[c] = \sum_{k \geq 1} k * \frac{9}{10^k} = \frac{10}{9} \approx 1.11 = \theta(1)$$

Resposta a **3.**

Custo amortizado

$$\hat{c} = c + \phi_{novo} - \phi_{antigo}$$

$$(\text{onde } \phi(s) = \tfrac{soma\ dos\ digitos}{9})$$

Vamos supor um exemplo para '23999' para '24000':

- Antes
$$\phi_{antigo}(s) = \frac{2+3+9+9+9}{9} = \frac{32}{9}$$
- Depois
$$\phi_{novo}(s) = \frac{2+4}{9} = \frac{6}{9}$$

Mudança de potêncial :

$$\phi_{novo} - \phi_{antigo} = \frac{6}{9} - \frac{32}{9} = -\frac{26}{9}$$

O custo real(**c**) foi **4**, pois foram $3 * (9- > 0) + 1 * (3- > 4)$ logo

$$\hat{c} = c + \phi_{novo} - \phi_{antigo} = 4 - \frac{26}{9} = \frac{10}{9}$$

De uma forma geral, se alterarmos $(k-1)$('9') + 1 último dígito $\neq 9$, o custo real será :

$$c = k - 1 + 1 = k$$

O potêncial diminui :

- cada $9- > 0$ baixa 9 dígitos ;
- o último dígito $\neq 9$ aumenta 1 ;
- No caso de $3999- > 4000 \implies +1 - 9 - 9 - 9 = -26 \equiv \frac{-26}{9}$ (visto anteriormente) .

Logo,

$$\Delta_\phi = \phi_{novo} - \phi_{antigo} = \frac{1 - 9 * (k-1)}{9} = \frac{10 - 9k}{9}$$

então,

$$\hat{c} = c + \Delta_\phi = k + \frac{10 - 9k}{9} = \frac{10}{9} = \theta(1)$$

Capítulo 9

Teste 2 - 24/25

9.1 I - Estruturas de Dados

MinHeap Considere uma min-heap com 12 elementos armazenada no seguinte array

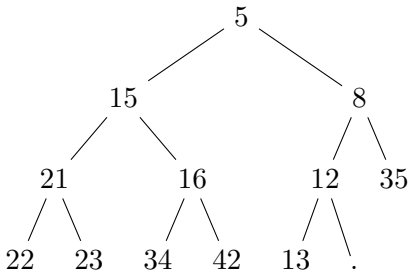
0	1	2	3	4	5	6	7	8	9	10	11
5	15	8	21	16	12	35	22	23	34	42	13

1. Qual a posição no array do valor 42 após a remoção (na min-heap) dos dois menores elementos?
2. Qual o valor do array na posição 3 se o elemento da min-heap na posição 8 (23) for alterado para 7?

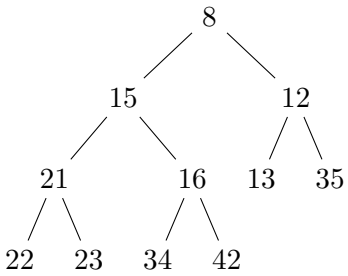
Resolução:

Resposta a **1.**

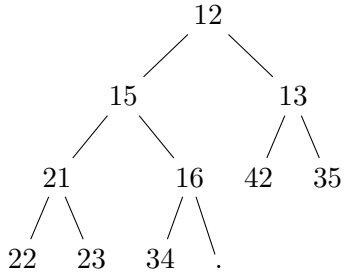
Min-heap (original)



Min-heap (remoção 5)



Min-heap (remoção 8)



Construindo de novo o array da min-heap (após as remoções) :

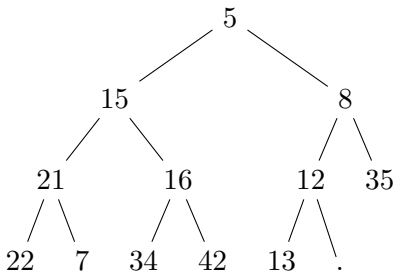
0	1	2	3	4	5	6	7	8	9
12	15	13	21	16	42	35	22	23	34

Assim posição no array do valor 42, após remoções é posição 5.

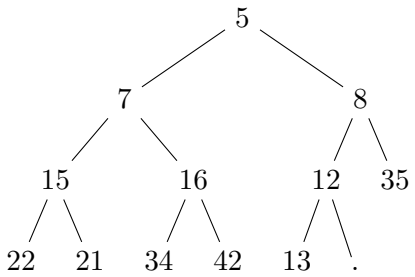
Resposta a **2.**

Se alterarmos o valor 23 para 7, na min-heap iremos ter de subir o valor na árvore, até achar uma raiz da qual o valor dessa raiz seja ≤ 7, logo :

Troca do valor 23 por 7



Após subir o valor 7

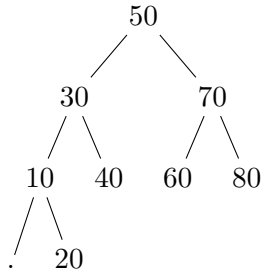


Construindo de novo o array da min-heap (após troca de valores) :

0	1	2	3	4	5	6	7	8	9	10	11
5	7	8	15	16	12	35	22	21	34	42	13

Assim o valor na posição 3 fica 15.

AVL Considere a seguinte AVL com 8 nodos e altura 4

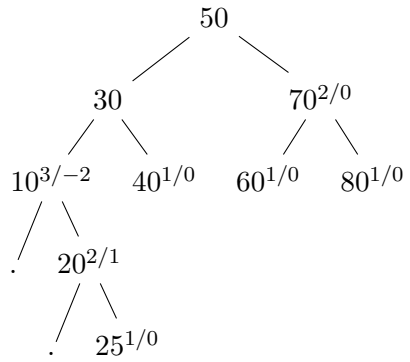


1. Depois de inserir nesta AVL os elementos 25 e 45 (por esta ordem), qual a alturada árvore resultante?
2. Das seguintes sequências de inserções na árvore, qual ou quais provocam uma mudança do nodo raiz da árvore? A: 75, 65, 55 ou B: 5, 45, 15?

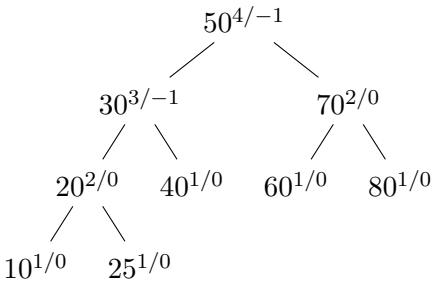
Resolução:

Resposta a **1.**

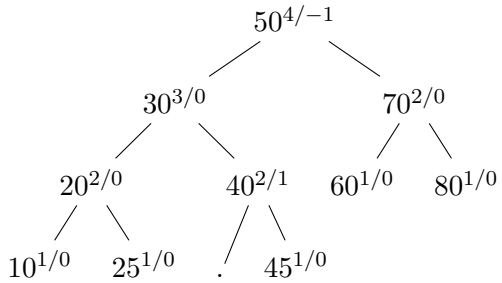
Inserir 25



(fica desequilibrado em **10**, logo rotação Right − > Left)



Inserir 45

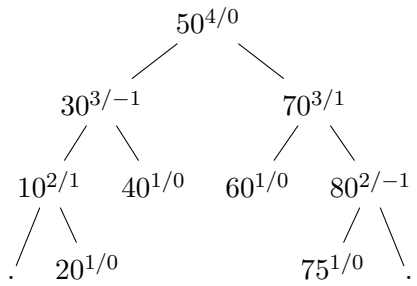


A árvore resultante fica com altura de 4.

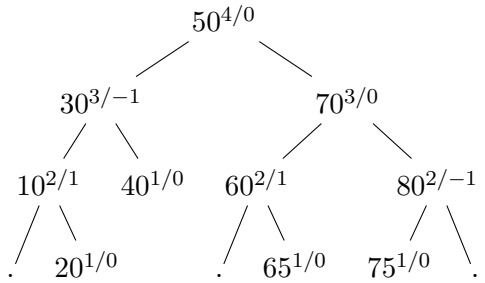
Resposta a **2.**

Opção A :

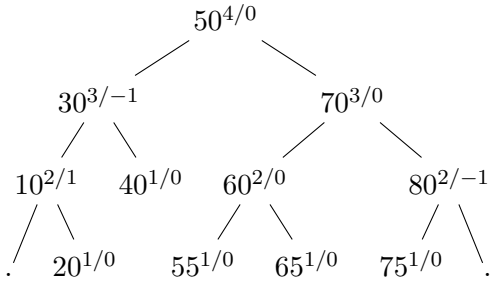
Inserir 75



Inserir 65



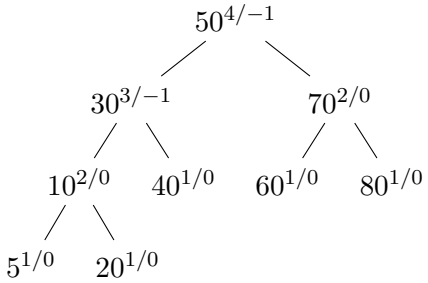
Inserir 55



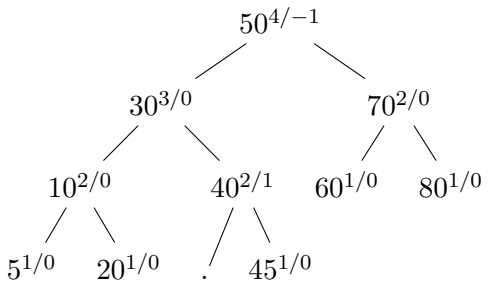
A árvore manteve a mesma raiz 50 .

Opção B :

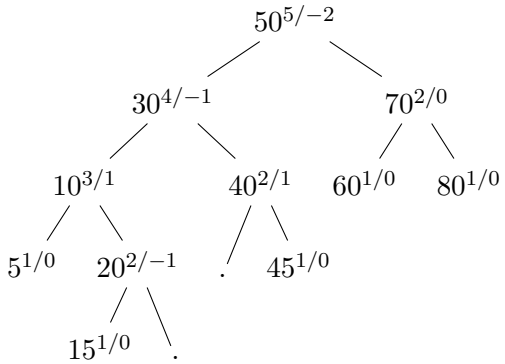
Inserir 5



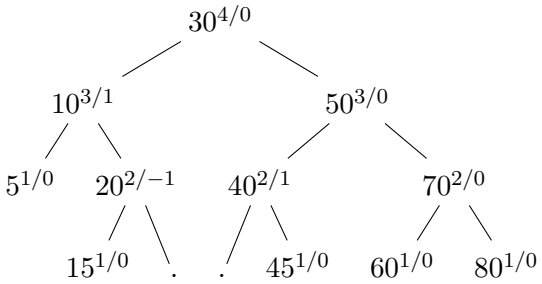
Inserir 45



Inserir 15



(fica desbalanceado em **50**, logo rotação Left $->$ Right)



A árvore na **sequência B** provoca mudança do nodo da raiz de $50 -> 30$.

THash Considere a seguinte tabela de hash de inteiros, implementada com *open-addressing* e *linear probing* num array com 10 elementos, onde a função de hash é o resto da divisão inteira ($hash(x) = x\%10$) e onde o estado de cada célula é Free, Deleted ou Used.

0	1	2	3	4	5	6	7	8	9
U	F	U	U	D	U	U	F	F	F
20	74	42	23	14	52	36	17	34	80

1. Em que posição seria feita a inserção do elemento 75?
2. A consulta do elemento 74 implica o acesso a quantos elementos do array?

Resolução:

Resposta a **1.**

$75 = 10 * 7 + \mathbf{5}$, ficaria na posição 5, mas como essa posição já se encontra ocupada, percorremos as próximas posições até acharmos uma célula vazia, neste caso :

0	1	2	3	4	5	6	7	8	9
U	F	U	U	D	U	U	U	F	F
20	74	42	23	14	52	36	**75**	34	80

Logo a inserção do elemento 75 será na posição 7 .

Resposta a **2.**

Para consultarmos o elemento 74, primeiro calculamos a posição onde se encontraria, neste caso

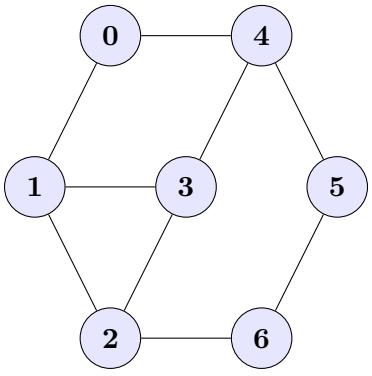
$74 = 10 * 7 + \mathbf{4}$, logo começamos a olhar para a posição 4 do array :

- $4 \implies D$, não está aqui (**1 acesso**) ;
- $5 \implies U(52)$, não está aqui (**1 acesso**) ;
- $6 \implies U(36)$, não está aqui (**1 acesso**) ;
- $7 \implies F$, quando encontramos um *FREE*, sabemos que não existe o elemento que procurávamos e paramos! (**1 acesso**) .

Logo, a consulta faz 4 acessos.

Nota : Mesmo que tenha o valor 74 representado no array, se a célula estiver assinalada como F(FREE)ou D(DELETED) é apenas lixo e não o valor real que procuramos.

Grafos Seja G o seguinte grafo não pesado e não orientado representado usando **listas de ajdacência ordenadas por ordem crescente** do destino.



1. Numa travessia Depth-First iniciada no vértice 0 qual o último vértice a ser visitado?
2. Numa travessia Breadth-First iniciada no vértice 3 qual o último vértice a ser visitado?

Resolução:

Resposta a **1.**

(Não esquecer que segundo o enunciado as **listas de adjacência estão ordenadas por ordem crescente**)

$$\{\mathbf{0}\}$$
$$\{\cancel{\mathbf{1}}, 4\}$$
$$\{\cancel{0}, \mathbf{2}, 3\}$$
$$\{\cancel{1}, \mathbf{3}, 6\}$$
$$\{\cancel{1}, \cancel{2}, \mathbf{4}\}$$
$$\{\cancel{0}, \cancel{3}, \mathbf{5}\}$$
$$\{\cancel{4}, \mathbf{6}\}$$

Logo,

$$DFS = \{0, 1, 2, 3, 4, 5, 6\}$$

o último vértice a ser visitado é o $\underline{6}$.

Resposta a **2.**

(Não esquecer que segundo o enunciado as **listas de adjacência estão ordenadas por ordem crescente**)

$$[\mathbf{3}]$$
$$[\cancel{\mathbf{1}}, 2, 4]$$
$$[\cancel{2}, 4, 0]$$
$$[\cancel{4}, 0, 6]$$
$$[\cancel{0}, 6, 5]$$
$$[\cancel{6}, 5]$$
$$[\cancel{6}, 5]$$
$$[\mathbf{5}]$$

Logo,

$$BFS = \{3, 1, 2, 4, 0, 6, 5\}$$

o último vértice a ser visitado é o $\underline{5}$.

9.2 II - Tabelas de Hash

Numa tabela de hash implementada usando *open address* e *linear probing*, o **probe-count** de cada chave define-se como o número de células que foram consultadas para inserir essa chave.

```
1  #define HSize ...
2  #typedef struct bucket {int status; int value} THash [HSize];
```

Defina a função *int max_probe (THash h)* que calcula o maior probe-count associado às chaves armazenadas na tabela h. Assuma que existe uma função *int hash (int k)* que calcula o hash de uma chave k.

Resolução:

Para cada elemento da hash :

- verificar se é Used ;
- calculcar a hash onde começam ;
  - se o valor da hash coresponder à posição atual, $probe = 1$;
  - se não, calcula a partir da sua hash até à posição onde ficou armazenado e compara para ver se é o maior.

```
1   int max_probe(THash h){
2
3       int maior = 0;
4       for(int i = 0; i < HSize; i ++){
5           if(h[i].status == USED){
6               int p = hash(h[i].value);
7               int probe = 1;
8
9               while(p != i){
10                  p = (p+1) % HSize;
11                  probe ++;
12              }
13
14              if(probe > maior)
15                  maior = probe;
16          }
17      }
18      return maior;
19  }
```

Notas extras : Em termos de complexidade, percorremos a tabela HSize e para cada elemento podemos percorrer quase a tabela toda, logo no pior caso

$$O(HSize^2)$$

9.3 III - Min-heap

Defina uma função *int toArray (ABin a, int h[], int N)* que, dada uma árvore binária de inteiros que satisfaz as propriedades das min-heaps a converte para um array (escrevendo no máximo N elementos).

Resolução:

Ter em consideração...

- Propriedade das min-heaps $->$ escreve da esquerda para a direita, logo como já é uma árvore sabemos que a $raiz->$ índice 0, $esquerda->2*i+1$ e $direita->2*i+2$;
- Como o enunciado diz "escrevendo no máximo N elementos", teremos de devolver na nossa função o sumatório($\sum$) dos elementos escritos .

```
int toArrayAux(ABin a, int h[], int i, int N){

    //ver se a arvore e nula e certificar que nao faz mais que N elementos
    if(a == Null || i >= N)
        return 0;

    h[i] = a->valor;

    //soma todos os elementos escritos
    return 1 + toArrayAux(a->esq, h, 2*i+1, N)+
               toArrayAux(a->dir, h, 2*i+2, N);
}

int toArray(ABin a, int h[], int N){
    return toArrayAux(a, h, 0, N);
}
```

9.4 IV - Grafos

Defina a função *int componentesOK (GrafoL g, int c[])* que, dado um grafo não orientado e um array com a identificação, para cada vértice, da componente ligada a que pertence, testa se esse array está bem construído. Por outras palavras, testa se para todos os vértices i e j, c[i]==c[j] sse há um caminho de i para j. Use para isso (se for esse o caso) as funções apresentadas nas aulas (DF, BF, Dijkstra, ou FW). Qual a complexidade da função apresentada?

Resolução:

Vamos ver primeiro a ideia geral da função e o que são componenetes ligados :

Vertices	0	1	2	3	4	5
c[]	0	0	0	1	1	2

A ideia é fazer um DFS a partir de todos os vertices, na qual :

- Mantemos um vetor visitados, para evitar repetir ;
- Todos os vértices encontrados pelo DFS têm de ter o mesmo valor de *c*[]. Se um desses vértices tiver outro identificador, então o array está errado ;
- Ao terminar o DFS, sabemos que todos os vértices dessa componente foram visitados ;
- Algo muito importante é garantir também que cada identificador aparece apenas uma vez numa DFS, para evitar casos de vértices isolados que passam a ter o mesmo identificador, mas não existe caminho entre eles ;

```
int dfsComp(GrafoL g, int v, int comp, int c[], int vis[]){

    vis[v] = 1;

```

```
5         //verifica se o vertice pertence a componente esperada
6         if(c[v] != comp)
7             return 0;
8
9         for(struct edge *e = g[v]; e != Null ; e = e->next){
10            if(!vis[e->dest]){
11                if(!dfsComp(g, e->dest, comp, c, vis))
12                    return 0;
13            }
14        }
15        return 1;
16    }
17
18    int componentesOK(GrafoL g, int c[]){
19        int vis[MAX] = {0};
20        int usado{MAX} = {0};
21
22        for(int i = 0; i < MAX; i++){
23            if(!vis[i]){
24                //verifica se ja apareceu outra componente com este identificador
25                if(usado[c[i]])
26                    return 0;
27
28                usado[c[i]] = 1;
29
30                if(!dfsComp(g, i, c[i], c, vis))
31                    return 0;
32            }
33        }
34        return 1;
35    }
```

De uma forma curta... Percorre o grafo com DFS. Sempre que encontro uma nova componente, verifico que todos os vértices visitados têm o mesmo identificador em $c[]$. Além disso, verifico que cada identificador corresponde apenas a uma componente.

Cada vértice é visitado uma única vez e cada aresta é percorrida uma única vez (duas num gráfo não orientado), logo

$$O(V + E)$$

Apêndice A

Notas de estudo

A.1 Pré-Condição

- O que tem de ser verdadeiro antes do algoritmo começar .

Nota : Quando nos referimos a *int a*[], numa pré-condição dizer que $a[]! = Null \neq a\ tem\ tamanho \geq 0$, pois *int a*[] é um apontador para o início do array, então quando dizemos

$$a[]! = NULL \equiv a[0]! = NULL$$

(o que não prova nada !)

A.2 Pós-Condição

- O que tem de ser verdadeiro quando o algoritmo termina ;
- Descreve o efeito final e garante que o objetivo foi atingido ;
- Ter em atenção ao nome da função, enunciado e último return, pois pode conter pistas ;
- Serve para descrever exatamente o efeito da função .
- Quando a função devolve um índice ou um comprimento, a pós-condição costuma ter uma estruta do genero :
  - Uma propiedade que é verdadeira para o resulta (Ex:. $soma \leq lim$) ;
  - Uma propiedade de maximizante (”não dá para aumentar mais”) ;
  - É precisamente a segunda parte (Ex:. $N = 0$ ou a proxima soma ultrapasso o lim) que garante uma correção correta(neste caso que o prefixo é o maior possivel) ;
- Sempre que um algoritmo devolve o primeiro, o último, o menor ou maior, a pós-condiçao precisa de uma propriedade de minimalidade/ maximalidade. Aqui ”primeiro”traduz-se como :

”Todos os indices anteriros satisfazem a igualdade”

A.3 Invariante

- A expressão que descreve uma propriedade que é verdadeira antes de cada interação e que está relacionada com o trabalho já realizado pelo ciclo .

$$\text{Invariante} + \text{Condição de saida} \implies \text{pós-condição (do ciclo)}$$

- Quando temos um algoritmo que vai acumulando uma soma, quase sempre o invariante é do genero :
  - "A variavel sum é a soma dos elementos já processado" $\implies i-1$;
  - Depois acrescentamos a propriedade que interessa (Ex:. $sum \leq lim$, $sum = total$, etc) ;
  - Exemplo :

$$sum = \sum_{k=0}^{i-1} v[k] \wedge sum \leq lim$$

- Sempre que tiver ciclos *while(condição) i++;* e estiver a procurar o primeiro que satisfaz uma condição :
  - INV: $0 \leq i \leq N \wedge \forall_{k<i}$(os elementos anteriores não satisfazem a condição)

A.4 Variante

- É sempre não negativo, diminui estritamente a cada iteração e garante que o ciclo termina ;
- Inteiro positivo ;
- O variante não tem de chegar a 0, necessariamente .

A.5 Complexidade

- Definir o n :
  - "O que cresce quando o input cresce ?"
- Operação dominante :
  - "Qual a operação que mais se repete ?"
    - comparações ;
    - acessos ao array ;
    - chamadas recursivas .
- Analisar ciclos :
  - for $->$ $O(N)$;
  - (2*)for $->$ $O(N^2)$;
  - while$(n>1)\{n=\frac{n}{2};\}->$ $O(\log_N)$ Ex:. BST ;
  - for + divide $->$ $O(N \log_N)$

A.5.1 Recursividade

- Sempre que vire uma função recursiva com duas chamadas recursivas, perguntar :

"As duas chamadas são executadas ?"

- Se sim $->$ somam-se : $T(..) + T(..)$;
- Se a funçao escolhe uma ou outra (como numa BST) $->$ fica apenas uma chamda na recorrência ;
- Quando o enuncidado diz "Conte o nº de operações aritméticas"tento sempre escrever uma expressão como $3(N-1)$ ou $4(N-1)$ antes de concluir com $\theta(N)$;
- Recursividade nunca é em paralelo, é como "parar no tempo", executar todas as recursões e só depois continua a função princípal .

A.5.2 Pior Caso

- Maior tempo possivel .

A.5.3 Melhor Caso

- Menor tempo possivel .

A.5.4 Caso Medio

- Tempo esperado considerando todas as entradas ;
- Metodo :
  1. Assuma inputs aleatórios ;
  2. Calcular o nº médio de iterações ;
  3. Simplificar para Big-O .
- Custo = soma das operações :

```
1        for( i = 0; i < n; i++){
2            x = x + 1; //O(1)
3        }
```

$$\begin{aligned} &1(\text{inicialização}) \\ &+ \text{n(comparações)} \\ &+ \text{n(incrementações)} \\ &+ \text{n(corpo)} \\ &= 3n + 1 = O(N) \end{aligned}$$

- (continuação)
  - No caso de recursão (no caso de ser apenas uma recursão)

$$T(N) = chamadas * custo\ por\ chamadas$$

  - $T(N) = T(N-1) + c \implies O(N)$
  - $T(N) = 2 * T(\frac{N}{2}) + N \implies O(N \log_N)$

- Sempre que tivermos caso médio:
  - "O que acontece sempre ?"(esse custo entra diretamente) ;
  - "Que parte só acontece às vezes ?"(essa é multiplicada pela probabilidade)
- Tendo em consideração o exercico 5.2 do Teste de 2023 :
  - Independentemente do array , a função :

```
1          for(int i=0; i<N; i++)
2              if (a[i]==1)b++;
```

  - Este ciclo custa sempre $\theta(N)$, logo este custo aparece sempre ;
  - Temos 2 possibilidades :
    - Caso 1 - existe apenas um 1

      return g(a,N), que tem custo $\theta(2^N)$
    - Caso 2 - não há exatamente um 1

      return 0, não há custo adicional relevante
  - Então o tempo depende do array :
    - Tempo = $\theta(N)$ se não chama g ;
    - Tempo = $\theta(N) + \theta(2^N)$ se chama g ;
  - Como não sabemos qual dos casos vai acontecer usamos a esperança..
  - Valor esperado :
    - $E[x] = \sum P_i * X_i$ $(Probabilidade * Custo)$;
    - No caso estudado existem dois resultados possíveis :

      $$E[T] = P(nao\ chama\ g) * \theta(N) + P(chama\ g) * (\theta(N) + \theta(2^N))$$

    - Como o custo $\theta(N)$ acontece nos dois casos, podemos factorizar, ou seja, a única parte que acontece às vezes é a única a ser multiplicada pela probabilidade, assim

      $$E[T] = \theta(N) + P(b = 1) * \theta(2^N)$$

Nota :

- 2^k existe apenas quando tem duas chamadas recursivas ;

Nota :

- Sempre que $T(N) = T(N-1) + N \implies \theta(N^2)$, acabamos sempre por ter um sumatório .

Notas sobre Sumatorios :

- $\sum_{i=1}^{N} 1 = N$
- $\sum_{i=1}^{N} i = \frac{N(N+1)}{2} \implies \theta(N^2)$
- $\sum_{i=1}^{N} i^2 = \frac{N(N+1)(2N+1)}{6} \implies \theta(N^3)$

A.5.5 Custo Real vs Custo Armotizado

- Custo efetivo de uma operação específica, naquele momento

$$custo\ real = (\text{nº total de operações realmente executadas})$$

- "Quanto trabalho esta operação faz agora ?"
- Exemplos :
  - Inserir num array cheio $-> O(N)$;
  - push normal $-> O(1)$;
  - pop $-> O(1)$;
  - resize$-> O(N)$;

A.5.6 Custo Amortizado

- Custo médio por operação normal (remoção, inserção, etc.) numa sequência longa, mesmo que algumas operações sejam caras ;
- "Operações cara são raras e pagas por muitas baratas ."
- Métodos para calcular o custo Amortizado :
  - Agregado
    1. Considerar m operações ;
    2. Soma o custo total ;
    3. Divide por m .
  - Acconting method
    1. vai guardar crédito com operações baratas ;
    2. o crédito paga as caras no futuro .
    * Quando o algoritmo caro só acontece depois de muitas operações baratas, cada operação barata "poupa"um pequeno crédito. Depois, quando a operação cara acontece, usa esses créditos para a pagar.
  - Ex:. push num array dinãmico
    * Cada push paga 3 unidades ;
    * Usa 1 agora ;
    * Guarda 2 ;
    * Quando há resize o crédito acomulado pagas as cópias

$$O(1) \implies \text{amortizado} \implies \text{nunca fica negativo}$$

  - Quando usar custo amortizado ? $\implies$ "Há operações caras ocasionalmente misturadas com muitas baratas ? $\implies$ Sim $\implies$ Amortizado ;
  - Ex:. array dinâmicos , stacks com push/pull , tabelas hash (refreshing) ;
  - Apesar de algumas operações terem custo real $O(N)$, o custo amortizado por operação é $O(1)$.

Quando nos dão a função potência para cálcular o custo amortizado ;

$$\hat{c} = c + \Phi_{novo} - \Phi_{antigo}$$

- $\hat{c}$ é o custo amortizado ;
- c é o custo real ;
- Φ_{novo} é o potencial ;
- Φ_{antigo} é o potencial ;

- Potencial é uma "energia acumulada";
- Sempre que tiver função potencial do tipo :

$$\phi = \frac{soma\ de\ algo}{constante}$$

1. o que acumulou ?
2. o que diminui ?
3. quanto mudou ?
4. e depois dividir pela constante (ver exercicio 8.4 Teste 1 24/25)

Notas :

- Se aparecer uma função que "propaga carries"como no 8.4. quase sempre o custo médio e amortizado são constante, mesmo que o pior caso seja $O(N)$.

A.6 Hash Tables

- Nova capacidade $\implies$ ter em atenção o enunciado ;
- Divisão inteira :

$$\frac{75}{10} \implies 75 = 10 * x + y$$

  (onde y representa o resto, e a posição onde o elemento será inserido)
- Quando em procura numa hash table, se vir uma célula FREE termina a pesquisa. Se for DELETE não termina e continua ;
- Normalmente cada posição do array não guarda apenas só o valor, mas uma estrutura .

```
1  typedef enum{FREE, USED, DELETED} State;
2
3  typedef struct{
4      int value;
5      State state;
6  }Cell;
```

- cell.state == FREE − > F ;
- cell.state == USED − > U ;

- cell.state == DELETED $->$ D ;
- Durante um Garbage Collection :

```
        for( i = 0; i < size; i ++){
            //ignora todos F e D
            if(table[i].state == USED){
                insert(newTable, table[i].value);
            }
        }
```

A.6.1 Garbage Collection

- Custo assintótico $->$ $\theta(N)$;
- Na open addressing, o garbage collection faz "desaparecer"os DELETED e re-insere apenas os elementos USED, como se a tabela fosse reconstituída :
  1. Percorre a tabela ;
  2. Ignora as células Deleted ;
  3. Re-insere os Used numa nova tabela (do mesmo tamanho) .

A.6.2 Inserção na Hash Table

- "**Em média**, quantas posições do array consulta essa inserção ?
  - Ter em consideração o range das possiveis posições, e ver para os casos que abrange, quantas consultas acontecem em cada célula para colocar o novo elemento ;
  - Se o invervalo cobre um nº de valores que é múltiplo do tamanho da tabela (como 100 a 199 , 0 a 99, 1000 a 1099), todos os restos são equiprováveis $\implies$ média simples ;
    - Todos os restos aparecem 10 vezes logo $\frac{Media}{10\ posicao}$.
  - Se não for o caso (ex:. 100 a 120), primeiro conta-se quantas vezes aparece cada resto e depois $\implies$ média ponderada ;
    - Exemplo :

Resto	Quantidade
0	3
1 a 9	2

    - Tendo em consideração os custos de consulta de cada posição :

$$\frac{3*1+2*1+2*1+...+2*1}{21}$$

      (onde multiplicamos o custo de cada posiçao pela sua quantidade, e dividimos por o nº possível de números)

- prob-count $->$ o nº de células consultadas quando a chave foi inserida .

A.7 Árvores

A.7.1 Árvore Binária de Pesquisa (BST)

- $Esquerda < Raiz < Direita$
- Se vir $m = \frac{N}{2}$, está a dividir o array ao meio $->$ normalmente para construir uma BST equilibrada (a partir do array ordenado) ;
- Se vir $2 * i + 1$ e $2 * i + 2 \implies$ está a seguir a representação da heap ;
- Todos os valores da esquerda são menores ;
- Todos os valores da direita são maiores ;
- Cada nó tem de respeitar os limites impostos por todos os seus antepassados, não apenas pelo pai ;
- "Só interessa manter a ordem ";
- Problema da BST
  - Pode ficar desequilibrada, afeta a procura ou seja $O(N)$;
- Diminui apenas uma unidade :
  - $T(N) = 2T(N-1) + O(1) \implies \theta(2^N)$
  - Típico de árvores com altura N (BST)

A.7.2 Heap(min/max)

- Heap(min/max)
  - completa ;
  - min-heap= $pai \leq filhos$;
  - max-heap= $pai \geq filhos$
- Se o enunciado apenas disser "heap sobre array" $\implies$ escrever só os elementos ;
- Se falar em "array dinâmico", é importante mostar as posições livres (_ ou H) e, se possível, indicar a capacidade do array.

A.7.3 Min-Heap

- Sempre é representada numa árvore da esquerda para a direita, quando se da insert, tem de se fazer bubble up (se for um valor menor que o anterior) ou bubble down(quando o valor é maior que os seguintes) quando se dá pop da raiz .
- As árvores semi-completas em array, têm os seus nodos armazenados por nível sempre da esquerda para a direita, logo num array da min-heap ;
- Índice onde se encontra a sub-arvore esquerda do nodo da posição i : $2i + 1$;

- Índice onde se encontra a sub-arvore direita do nodo da posição i : $2i + 2$;
- Índice onde se encontra o pai do nodo da posição i : $\frac{(i-1)}{2}$;
- Min-Heap é uma árvore binária em que cada nodo é menor ou igual a todos os seus sucessores ;
- Uma árvore diz-se semi-completa se todos os níveis da árvore estão completos, com a possível exceção do último, que pode estar parcialmente preenchido .

A.7.4 Árvore equilibrada (AVL)

- $|altura\ dir - altura\ esq| \leq 1$
- Todos os valores da esquerda são menores ;
- Todos os valores da direita são maiores ;
- Para cada nó $|altura\ dir - altura\ esq| \leq 1$
- Este género de árvores ajuda a procura, inserção e remoção a demorar sempre $O(\log N)$;
- "Quero manter a ordem e manter a árvore equilibrada "
- Divide o problema a meio :
  - $T(N) = 2T(\frac{N}{2}) + O(1) \implies \theta(N)$;
  - Típico de árvores de altura $\log N$ (AVL).

Nota importante : Quando se tem $int\ v[]$, isto significa que ele ta a apontar para $v[0]$, $v == \&v[0]$ logo $v + k =$ endereço de $v[k]$, ou seja $v + m + 1 == \&v[m + 1]$.

- Percorrer
  - Pre-order = Nó− >Esq− >Dir ;
  - Inorder = Esq− >Nó− >Dir ;
  - Post-order = Esq− >Dir− >Nó
- Complexidade

BST (genérica)	Melhor	Pior	Caso Médio
procura	$\theta(1)$	$\theta(N)$	$\theta(\log N)$
insert	$\theta(1)$	$\theta(N)$	-
remove	$\theta(1)$	$\theta(N)$	-

(Heap/Array) BST equilibrada (AVL)	Melhor	Pior	Caso Médio
procura	$\theta(1)$	$\theta(\log N)$	-
insert	$\theta(1)$	$\theta(\log N)$	-
remove	$\theta(1)$	$\theta(\log N)$	-
ver minimo/maximo	$\theta(1)$	$\theta(\log N)$	-

Nota importante : Árvore completa $- >\ \theta(\log N)$.

- Sempre que a pesquisa numa árvore segue apenas um ramo (como BST ou pesquisa binária), a recorrência costuma ser da forma $T(N) = T(\frac{N}{2}) + O(1)$ e a resposta é quase sempre

$$\theta(\log N)$$

- Exemplo : Para $N = 8$, a cada chamada o custo é 2 e $\frac{N}{2}$

$$\begin{aligned} T(8) &= T(4) + 2 \\ &= T(2) + 2 + 2 \\ &= T(1) + 2 + 2 + 2 \end{aligned} \tag{A.1}$$

  Como $T(1) = 2$ então $2 + 2 + 2 + 2 = 8$

$$2\log_2(8) + 2 = 2\log_2(2^3) + 2 = 2 * 3 + 2 = 8$$

- Sempre que tiver uma recorrência $T(N) = T(\frac{N}{2}) + k$, onde k é uma constante $(1, 2, ..., 5)$ a solução é

$$T(N) = k\log_2 N + \frac{cosntante}{k} = \theta(\log N)$$

- O nº que multiplica o $\log N$ é precisamente o custo constante de cada nível da recursão .

A.7.5 Array para Árvore

- "O array contem os elementos por ordem crescente ou contem a representação da árvore ?"
  - se contem a representação, usar $2i + 1$ e $2i + 2$ (Percorrer) ;
  - se contem apenas os valores ordenados, usar $\frac{N}{2}$ (Consutir a árvore) ;

A.7.6 ABin

- Uma árvore binária qualquer, não tem nenhuma propriedade especial;

```
1        typedef struct nodo {
2            int valor;
3            Struct nodo *esq ;
4            Struct nodo *dir ;
5        }*Abin
```

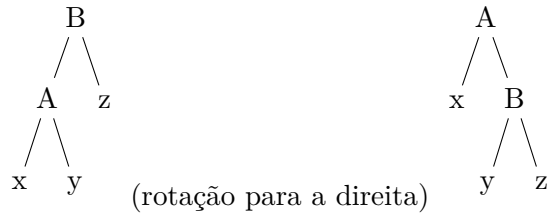
A.7.7 Rotações da Árvore

Nota :

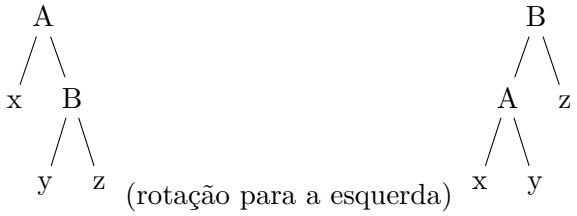
$$Raiz^{Altura/Fator\ Balanco}$$

onde sempre que o Fator de balanço(**altura dir - altura esq**) for diferente de $-1, 0, 1$, significa que temos de fazer rotação nesse nodo/raiz.

Left $->$ Right



Right $->$ Left



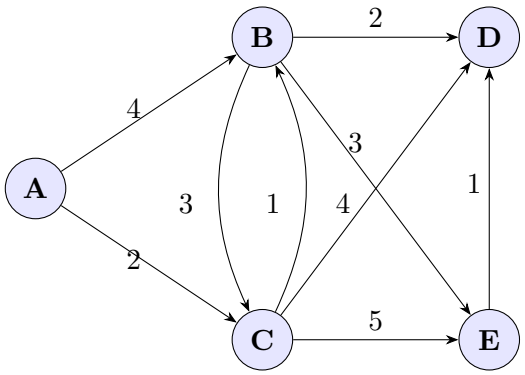
- (continuação)
  - inserir um novo elemento :
    - para cada nodo se $k < node \implies left$;
    - para cada nodo se $k > node \implies right$.
- Esq $->$ esquerda mais alta (-1) ;
- Dir $->$ direita mais alta (1) ;
- Bal $->$ equilibrada (0) ;

A.8 Algoritmos de Pesquisa

- Calcular a distância num grafo não pesado $\implies$ BFS/DFS ;
- Calcular a distância num grafo pesado (e orientado) $\implies$ Dijkstra .

A.8.1 Dijkstra's

- Caminho mais curto + somar os valores ;
- Tem-se em consideração a escolha global, não apenas a partir do nó em que estamos ;

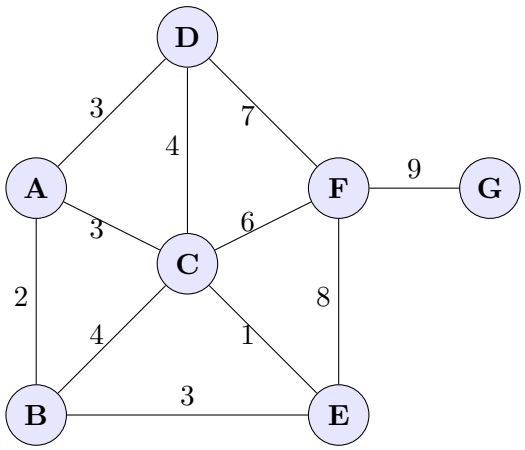


$$A, C, B, D, E$$

- $O(|E| + |V| \log |V|)$
- *dijkstra(g, vertice onde começa(vi), pais , pesos)* ;
  - *peso[v]* = distância mínima de vi a v ;
  - *pais[v]* = vértice anteriror a v no caminho mínimo percorrido .
- "Qual o *pais[v]* ?"
  1. encontrar caminho mínimo de vi a v ;
  2. olhar para o vértice imediatamente antes do v ;
  3. esse vértice pai *pais[v]* .
- para cada vértice, guarda sempre o melhor caminho conhecido da origem;
- se o vértice não tiver uma aresta a ligar ao pai, o vértice nem sequer é candidato ;

A.8.2 Prim's

- Minimum Spaming Tree = menor peso + sem ciclos ;
- Quando usamos o Prim, não olhamos só para os vizinhos do último nó adicionado, mas para os vizinhos de todos os nós que já estão na árvore ;
- Se tiver empate de peso $\implies$ escolha arbritária ;



$$A, B, E, C, D, F, G$$

- Matriz de Adjacência $O(V^2)$;
- Binary Heap $O(V \log V + E \log V)$;
- Na resposta é bom incluir a tabela + árvore final (ver exercício 2.7 IV - Grafos (b)) ;
- Seja S os vértices já na árvore, escolhemos sempre a aresta de menor peso que liga S ao resto .

A.8.3 BFS

- Começa no vértice x ;
- Visita vizinhos (ter em atenção o enunciado, pois a lista de adjacência pode ter regras) ;
- Insere vizinhos numa fila (Queue) ;
- Marcar nos vizitados ;
- Ver exercicio (7.3 III - Grafos 1.)

A.8.4 DFS

- Começa no vértice x ;
- Visita vizinhos (ter em atenção o enunciado, pois a lista de adjacência pode ter regras) ;
- Pilha (não guarda os vizinhos antigos, apenas o do vértice atual que estamos) ;
- Se todos da pilha tiverem visitados volta para o anterior ;
- Ver exercicio (7.3 III - Grafos 2.)
- Se o enunciado falar em "maior caminho, descida, subida,etc", normalmente usa-se DFS + memorização (programação dinâmica) ;
- Se o enunciado fala em "figura, ilha, componente conexa,etc..", usa-se DFS /BDS com uma matriz de visitados ;

A.8.5 DFS vs BFS

- Sempre que fizermos um DFS ou BFS, num grafo representado por lista de adjacência, a complexidade é quase sempre :

$$\theta(V + E)$$

- Se o grafo estiver representado por uma matriz de adjacência, então a compelxidade é

$$\theta(V^2)$$

porque para cada vértice teria de percorrer uma lista inteira da matriz .

A.8.6 Notas de Grafos

- Sempre que :
  - "Existe um caminho de x para y ?";
  - "x pertence a um ciclo ?";
  - "É possivel voltar ao vértice inicial ?".
- A solução começa por :

```
int vis[MAX]={0};
dfs(...)
```

- Cria um vetor para guardar visitados (Ex:.$[0,0,0,0]$, onde 0 é não visitado e 1 é visitado) ;
- A única coisa que muda é a condição de paragem dentro do dfs (Ex:. *(e− > dest == origem; v == y; ...)*) (Ver 2.8 de Exame 2024)

A.9 Notas Extra

- Em c, o || faz "short-circuit", ou seja quando temos *retunr (...)||(...)* sabemos que se a primeira chamada devolver 1, a segunda nem sequer é executada ;

Notas :

- $a \wedge b$ é verdadeira(1) apenas para $a = 1 \wedge b = 1$;
- $a \vee b$ é falso(0) apenas para $a = 0 \vee b = 0$;

A.9.1 Fibonacci

```
int fibonacci(int n) {
int a = 0, b = 1;

for (int i = 0; i < n; i++) {
    int temp = a + b;
    a = b;
    b = temp;
}

return a;
}
```

- Melhor caso $- >$ $\theta(n)$, o ciclo executa n iterações, mesmo quando $n = 0$ o custo é constante.
- Pior caso $- >$ $\theta(n)$, para uma entrada de tamanho n, são executadas n iterações.
- Caso Médio $- >$ $\theta(n)$, o ciclo executa exatamente n iterações.
- Pré-Condição $- >$ $n \geq 0$, n tem de ser um inteiro não negativo .
- Pós-Condição $- >$ $return = F(n)$, ou seja, função devolve o n-ésimo número de Fibonacci.
- Invariante $- >$ $a = F(i) \wedge b = F(i+1)$ onde :
  - a contém o número de Fibonacci correspondente a i;
  - b contém o número de Fibonacci correspondente a i+1.
- Vairante $- >$ $n - i$
- Extra:

– Função Fibonacci (iterativa) $\implies \theta(n)$ (complexidade temporal) ;
– Função Fibonacci (recursiva) $\implies \theta(2^n)$ (complexidade temporal no pior caso) ;